Pose Estimation via Visual Inertial Wheel Odometry in Autonomous Vehicles

Bachelor Thesis

Author: Dimitrios Asimakopoulos
Supervisor: Prof. Panos Trahanias

Department of Computer Science
University of Crete
Heraklion, Greece

Abstract

Accurate and robust vehicle localization is a cornerstone of safe and reliable autonomous driving, yet many research solutions remain confined to idealized settings or rely on complex and inaccessible software stacks. This work presents an easy-to-deploy odometry module that brings together wheel encoders, an IMU sensor, and a camera-based traffic-sign detector into a single cohesive system. By packaging all components as standard ROS nodes and Gazebo plugins, the module can be tested in simulation and transferred to real vehicles with minimal changes.

At the core of the implementation we have a lightweight Extended Kalman Filter that continuously blends motion estimates from wheels and IMU with occasional landmark updates whenever a known traffic sign enters view. This design not only curbs the gradual drift typical of dead-reckoning but also handles sensor noise and temporary dropouts gracefully, ensuring the vehicle maintains an accurate pose estimate even in challenging environments.

Extensive simulation trials demonstrate that this approach significantly reduces cumulative position error compared to wheel-only baselines, while sign-based updates prevent long-term additive bias. The resulting framework is modular, robust, and ready for real-world adaptation, offering a practical path toward more reliable localization in next-generation autonomous vehicles.

Keywords: Autonomous Driving, Visual Inertial Wheel Odometry, Sensor Fusion, Simulation, Landmark Detection of Traffic Signs, AI Object Recognition, Pose Estimation.

Declaration of Authorship

I, Dimitrios Asimakopoulos, affirm that this thesis, titled 'Pose Estimation via Visual Inertial Wheel Odometry in Autonomous Vehicles', is an original work that reflects my own research and findings.

- This thesis was conceived and completed during my candidature for a research degree at this university.
- Any previously submitted portions of this work were explicitly stated as such.
- Quotes from external sources are accompanied by clear attributions.
- With the exception of quoted material, this thesis is entirely the product of my own efforts and creativity.
- I also acknowledge any significant contributions, guidance, or support received during the research process. When collaborating with others on aspects of this work, I have explicitly outlined their specific contributions and my own role in the project.

Signature: Dimitrios Asimakopoulos

Date: June, 2025

Acknowledgments

I am deeply grateful to my supervisor, Prof. Panos Trahanias, for his continuous support, invaluable guidance and feedback throughout this research journey.
I would also like to extend my thanks to Michael Maravgakis for their instructions and helpful advices.
I acknowledge Computer Vision and Robotics Laboratory of FORTH for providing the necessary resources to conduct my work.
This work would not have been possible without all of you.

Contents

1 Introduction **1**
1.1 Motivation and Problem Statement 1
1.2 Objectives and Contributions 1
1.3 Thesis Organization 2

2 Related Work **4**

3 Useful Tools **6**
3.1 Robotic Operating System 6
3.2 Gazebo 6
3.3 YOLO 7
3.4 Tool Utilization and Integration 8

4 Vision and Perception **9**
4.1 Overview of Semantic Landmark Observations 9
4.2 YOLOv5-Nano for Real-Time Detection 9
4.3 Custom Traffic-Sign Dataset 11
4.4 Detection Pipeline Implementation 12
4.4.1 Color Mask Preprocessing and Contour Extraction . . 13
4.4.2 Geometric Contour Selection 13
4.5 Camera Calibration and the Pinhole Model 14
4.6 3D Pose Estimation via PnP 14
4.6.1 Contour-based PnP and Fallback 15
4.7 Mapping Detections to Global Landmarks 16
4.7.1 Data Association and Mahalanobis Gating 16
4.7.2 Measurement Model Formulation and Jacobian 16
4.8 Performance and Robustness Considerations 17
4.9 Summary 17

5 Inertial and Wheel Odometry **18**
5.1 Introduction and Motivation 18
5.2 Gazebo Wheel Encoder Plugin 18
5.3 IMU Plugin 22
5.3.1 Plugin Architecture and SDF Integration 23
5.3.2 Custom IMU Message Format 23
5.3.3 Orientation Extraction and Kinematics 24
5.3.4 Noise, Drift, and Bias Considerations 24
5.3.5 ROS Subscription and Preprocessing 25
5.4 Summary 25

6 Sensor Fusion **26**
6.1 State Vector and Initialization 26
6.2 Process Model (Prediction Step) 27
6.3 Measurement Models 28
6.3.1 Wheel-Encoder Odometry 28
6.3.2 IMU Yaw Measurement 29
6.3.3 Vision-Based Landmark Updates 29
6.4 Calibration, Noise Tuning, and Extrinsics 31
6.5 Implementation Details and Pseudocode 32
6.6 Summary 33

7 TF and Coordinate Frames **35**
7.1 Frame Graph 35
7.2 Static and Dynamic Transform Broadcasting 37
7.3 EKF-Published Transform 37
7.4 TF Tree Validation 37
7.5 Coordinate Conventions and Consistency 38
7.6 Summary 38

8 Visualization and Monitoring **40**
8.1 RViz Usage 40
8.2 Overlaying Vision Detections via Markers 41
8.3 rosbag Recording and rqt Monitoring 42
8.4 Robot Localization Baseline Comparison 42
8.5 Summary 44

9 Evaluation and Results **45**
9.1 Evaluation Methodology 45
9.2 Sensor Characterization for Noise Tuning 46
9.3 Key Metrics and Improvements 46
9.4 Visualization of Results 47
9.4.1 Trajectories Comparison 47
9.4.2 State Components vs. Time 48
9.4.3 Error vs. Time 50
9.5 Analysis and Interpretation 52
9.5.1 Position Error Analysis 52
9.5.2 Yaw Error Analysis 52
9.5.3 Temporal Behavior and Drift Patterns 53
9.6 Limitations 53
9.7 Summary 53

10 Conclusion **54**
10.1 Limitations 55
10.1.1 Simulation vs Reality 55
10.1.2 Sensor Limitations 55
10.1.3 Algorithmic Assumptions 55
10.1.4 Scope Constraints 56
10.1.5 Impact of Limitations 56
10.2 Future Work 56
10.2.1 Algorithmic Enhancements 56
10.2.2 Additional Sensors 56
10.2.3 Real-World Testing 57
10.2.4 Scalability to Complex Environments 57
10.2.5 Code and Tooling Improvements 57

1 Introduction

1.1 Motivation and Problem Statement

Accurate vehicle localization lies at the heart of any autonomous driving system, enabling safe navigation through complex urban and highway environments. In real-world settings, GPS signals can be intermittent, and relying solely on wheel odometry or inertial measurements leads to unbounded drift over time. Simulation platforms such as ROS Noetic with Gazebo 11 offer a controlled yet realistic development environment: sensor plugins emulate IMU readings, wheel-encoder data, and a forward-facing camera, with configurable noise models. This setup allows for rapid algorithmic iteration without risk to hardware or public safety, while preserving the ability to transfer working configurations to actual vehicles with minimal reconfiguration.

Despite the availability of polished sensor-fusion packages like ROS's `robot_localization`, many researchers and engineers find these tools opaque and difficult to extend for novel measurement types. This thesis addresses that gap by developing a bespoke Visual-Inertial-Wheel Odometry (VIWO) framework. A central component is a custom Extended Kalman Filter (EKF), implemented in both C++ and Python to allow full visibility into algorithmic steps and parameter tuning. Beyond blending traditional motion cues—wheel-encoder speeds and inertial measurements—the filter leverages intermittent, high-confidence corrections whenever a YOLO-based detector recognizes a traffic sign whose map position is pre-defined. These semantic observations act as virtual waypoints, anchoring the pose estimate and preventing gradual drift inherent in dead-reckoning.

1.2 Objectives and Contributions

The objectives of this work are the following.

- **Design and implement a modular EKF:** Fuse wheel-encoder, IMU, and vision-based sign detections, with transparent code in C++/Python.
- **Integrate real-time YOLO sign detection:** Develop a ROS node with YOLOv5-Nano trained on a custom dataset, publishing detections for pose correction.
- **Setup Gazebo simulation scenarios:** Build a Gazebo world populated with traffic signs at precise map locations, configure realistic sensor noise, and provide launch files for reproducible tests.

- **Evaluate drift, accuracy, and resilience:** Perform extensive simulation trials (straight-line, curved, and sign-rich paths), compare against wheel-only and wheel+IMU baselines, and quantify position and heading improvements.

These contributions yield a modular, transparent VIWO framework in ROS/ Gazebo, demonstrating how semantic landmarks can be integrated into EKF-based localization to reduce drift and increase robustness.

1.3 Thesis Organization

The remainder of this thesis is structured to guide the reader through background, implementation, evaluation, and reflection on the VIWO framework. Chapter 2 reviews related work in visual odometry, visual-inertial fusion, semantic landmark localization, and ROS/Gazebo-based sensor-fusion frameworks, positioning our contribution within existing literature. Chapter 3 describes the software and simulation environment, detailing how ROS Noetic, Gazebo 11, and the YOLO framework are combined in our pipeline. In Chapter 4, we focus on vision and perception, explaining the traffic-sign detection pipeline: selection and training of the YOLOv5-Nano model, node structure, pose estimation via PnP, and publication of semantic observations. Chapter 5 covers inertial and wheel odometry by examining the Gazebo plugins for wheel encoders and IMU, including their architecture, message formats, kinematic models (Ackermann to twist), and noise and timestamp considerations. Chapter 6 presents the sensor-fusion EKF design, defining the state vector, process and measurement models, Jacobian derivations, noise tuning, and the ROS node implementation. In Chapter 7, we discuss TF and coordinate frames, describing the frame hierarchy, static and dynamic transform broadcasting, TF-tree validation, and coordinate conventions that ensure consistency across sensors. Chapter 8 addresses visualization and monitoring, covering RViz configuration, marker-based visualization of detections, rosbag recording, rqt monitoring, and a comparison against a baseline EKF (robot_localization). Chapter 9 outlines the offline evaluation and results methodology, including data collection, time alignment, error-metric computation, trajectory overlays, error plots, and interpretation (with placeholders for actual results to be filled later). Finally, Chapter 10 combines a critical examination of limitations and future work with the conclusion: it discusses assumptions (simulation vs. reality, sensor limitations, algorithmic scope), proposes directions for enhancement such as additional sensors, real-world testing, and scalability, and then summarizes the problem and the developed solution, highlights key findings (with placeholders for quantitative results),

reiterates contributions, and reflects on broader impact and next steps. This progression ensures a coherent narrative from the foundational tools and concepts, through implementation details and empirical evaluation, to critical reflection on and synthesis of the VIWO framework.

2 Related Work

Visual Odometry (VO) and Visual-Inertial Odometry (VIO) form fundamental building blocks in robot state estimation. Early filter-based VIO approaches—such as ROVIO—use an iterated Extended Kalman Filter to fuse IMU readings with photometric image-patch residuals, achieving real-time performance on resource-constrained platforms without explicit feature matching. Seminal work on MSCKF-based VIO [12] demonstrated the benefits of multi-state constraint Kalman filtering, while more recent monocular VIO systems like VINS-Mono [13] perform tightly-coupled fusion of pre-integrated IMU data and visual feature observations within a sliding-window framework, enabling centimeter-level accuracy and, when extended, loop-closure for global consistency. Reviews of VO and VIO techniques highlight both filter- and optimization-based paradigms and their trade-offs [1, 2].

Building on VIO, Visual-Inertial-Wheel Odometry (VIWO) further constrains planar motion by incorporating wheel-encoder measurements. Lee et al. [3] present a sliding-window MSCKF-based VIWO with online calibration of wheel-encoder intrinsics/extrinsics, modeling 2D odometry alongside IMU cues and identifying degenerate motions. Related work on Partial Invariant EKF for VIWO embeds rotation and velocity states on a Lie group to leverage kinematic constraints and dynamic outlier rejection for improved consistency and accuracy [4, 18]. Other works consider slip compensation and dynamic feature elimination to further enhance robustness in ground-vehicle contexts [5]. Geneva et al. [23] explore feature-based VIWO approaches, reinforcing the value of combining visual, inertial, and wheel data.

Extended Kalman Filters remain popular in robotics sensor fusion due to computational efficiency and transparent parameter tuning. High-precision EKF-based VIO variants (e.g., MSCKF 2.0) demonstrate that careful observability analysis and online calibration yield superior accuracy compared to smoothing methods [2]. Foundational texts on Kalman filtering detail these methods [21]. In ROS ecosystems, the widely used `robot_localization` package provides generic EKF/UKF nodes for fusing arbitrary sensor streams (IMU, wheel odometry, GPS, etc.), but treats measurement models as black boxes, motivating bespoke EKF implementations for deeper insight and custom landmark updates [6, 22].

Deep learning–based perception has also been integrated into odometry pipelines via semantic landmarks. Real-time object/detection networks such as YOLO [14, 15] achieve high mAP on traffic-sign benchmarks (e.g., GTSDB, TT100K) and are suitable for embedded automotive platforms. When fused as occa-

sional "anchor" measurements, semantic detections provide drift-correcting waypoints in VIWO systems [7, 8]. Pose estimation from detected contours often employs EPnP and RANSAC for robust 3D localization [16, 17].

Simulation environments like Gazebo coupled with ROS Noetic provide flexible, high-fidelity platforms for development and testing of VIWO systems. Prior works on simulator frameworks discuss Gazebo's design and paradigms [20], leveraging sensor plugins and physics engines for repeatable experiments before hardware deployment [9, 10]. Tight integration with ROS topics and TF transforms enables seamless transfer of code from simulation to real robots.

Despite this rich literature on VO, VIO, and VIWO, relatively few open-source works fuse wheel encoders, IMU, and semantic camera detections (e.g., traffic-sign landmarks) within a transparent EKF framework, with full ROS/Gazebo implementations tailored to ground-vehicle scenarios, realistic noise models, and landmark layouts. This thesis addresses these gaps by delivering a modular, extensible VIWO EKF pipeline—open-source and pedagogically clear—suitable for simulation and eventual real-world adaptation.

3 Useful Tools

In this section, we will provide a brief explanation about the tools that were used throughout this thesis while also highlighting their importance for the implementation of Pose Estimation via Visual Inertial Wheel Odometry in Autonomous Vehicles.

3.1 Robotic Operating System

ROS Noetic, running on Ubuntu 20.04, serves as the primary middleware for our VIWO framework, offering a standardized publish–subscribe architecture, rich message definitions and a mature ecosystem of tools. Among these, RViz provides an interactive 3D interface for visualizing the vehicle model, live TF frames, sensor streams, and estimated trajectories. In this project, RViz is configured to display the ground-truth path from Gazebo, the EKF's fused odometry (`/ekf/odom`), and detected traffic-sign landmarks—allowing real-time inspection of filter performance and immediate debugging of coordinate-frame or data-flow issues.

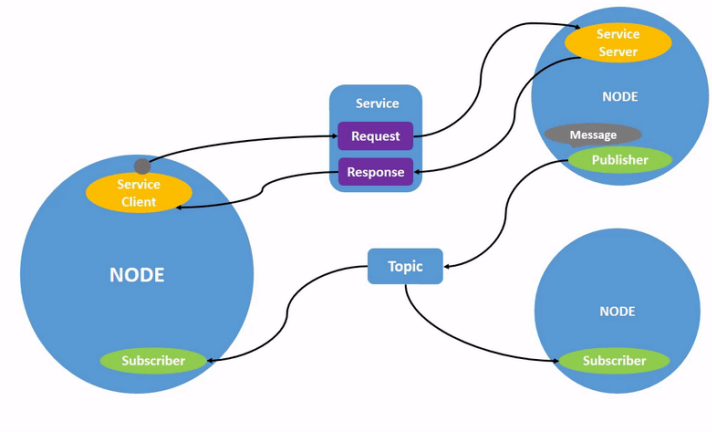


Figure 1: ROS Communication Graph

3.2 Gazebo

Gazebo 11 offers a high-fidelity physics simulator with realistic modeling of friction, gravity, and collisions, compatible out of the box with ROS Noetic through dedicated sensor and robot plugins. We leverage Gazebo to emulate an Ackermann-style vehicle equipped with a BNO055 IMU plugin, custom wheel-encoder plugin, and a Gazebo camera sensor. The world is populated with traffic-sign models placed at known coordinates, and sensor noise

parameters are tuned to mirror real-world characteristics. Using Gazebo ensures safe, repeatable testing of complex maneuvers—straight segments, curves, and sign-rich scenarios—before any hardware deployment. For this project, the map from the Bosch Future Mobility Challenge [11] will be used to simulate a city environment with road markings, crosswalks, roundabouts and parking spaces.

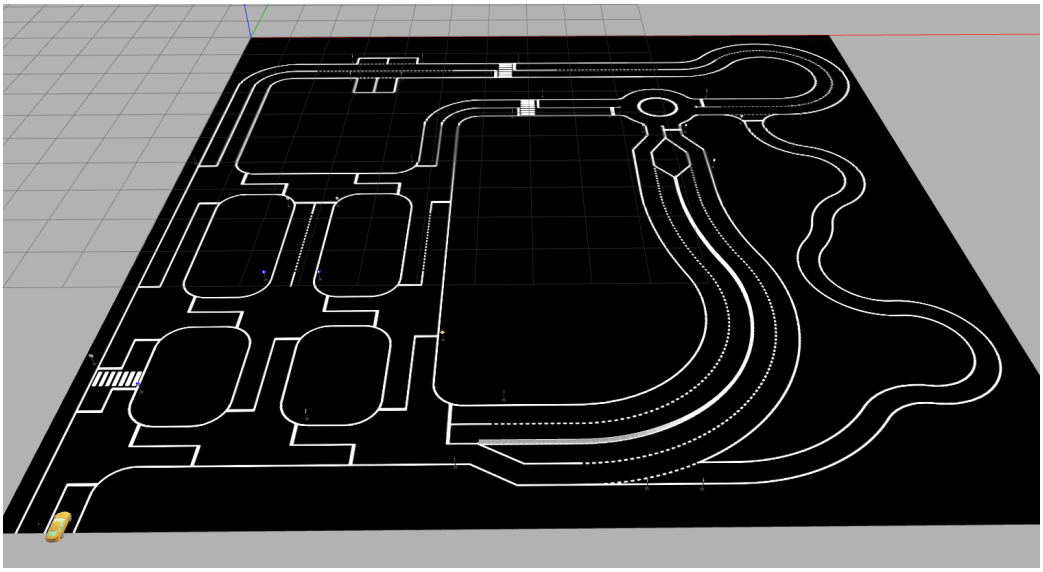


Figure 2: BFMC Gazebo Map

3.3 YOLO

For semantic perception, we integrate an Ultralytics YOLOv5-Nano model within a Python ROS node that performs real-time inference on camera images and publishes recognized sign poses. The detector handles preprocessing (e.g., cropping, undistortion) and applies shape-and-size priors for 3D pose estimation, emitting high-confidence "anchor" measurements to the EKF. Packaging YOLO as a ROS node makes it trivial to swap network variants or enable GPU acceleration, while maintaining consistency with ROS parameter conventions. Detailed discussion of the subscription topics and launch configuration is provided in Chapter 4.

3.4 Tool Utilization and Integration

All components are orchestrated through ROS launch files and parameter YAMLs that instantiate the Gazebo world, sensor plugins (IMU, wheel encoder, camera), the YOLO-based detection node, and the EKF fusion node in a reproducible manner. Topics and parameters follow standardized naming conventions described in their respective implementation chapters, so that swapping components or adapting to a physical robot involves minimal changes to the same launch infrastructure. RViz and rqt tools are launched alongside for real-time visualization of TF frames, trajectories, detection markers, and filter covariances, while `map_server` and recorded rosbag playbacks support offline analysis. Parameter YAML files manage sensor noise settings, filter covariances, and YOLO hyperparameters, centralizing configuration and ensuring consistency across simulation and deployment.

4 Vision and Perception

In this chapter, we detail the design and integration of a real-time traffic-sign detection pipeline within the VIWO framework, transforming semantic observations into robust pose-correction anchors. We employ a lightweight YOLOv5-Nano model trained on a custom eight-class dataset, paired with precise camera calibration and enhanced contour-based PnP pose estimation to translate image detections into measurement updates for the EKF. Through simulation experiments, we demonstrate that these semantic cues significantly curb odometry drift and improve global consistency in sign-rich driving scenarios.

4.1 Overview of Semantic Landmark Observations

In robotics, vision and perception serve as the eyes and interpretive layer that transform raw sensor data into actionable understanding of the environment. Traffic signs act as clearly defined, human-designed landmarks with known map coordinates and physical dimensions. Unlike generic geometric features that require complex matching, each sign detection provides both a class label (e.g., "STOP," "CROSSWALK") and size, enabling straightforward 2D–3D correspondences. When wheel slip or IMU drift causes dead-reckoning to deviate, a correctly recognized sign immediately re-anchors the filter to its true location on the map, bounding cumulative error. Moreover, semantic detections remain reliable under moderate lighting or scene changes, providing robust corrections where feature-based methods may fail. This synergy between low-level odometry and high-level perception underpins a more resilient and adaptive localization solution.

4.2 YOLOv5-Nano for Real-Time Detection

YOLOv5-Nano was selected as the core detection engine for its favorable trade-off between model compactness and inference speed. With a model size under 10 MB and a streamlined architecture optimized for edge devices, YOLOv5-Nano can achieve end-to-end detection times of less than 15ms per frame on a standard CPU, enabling real-time processing at 30Hz without requiring dedicated GPU acceleration. This performance envelope aligns well with the computational constraints of embedded automotive platforms and

ensures that vision-based updates remain timely within the high-frequency EKF loop.

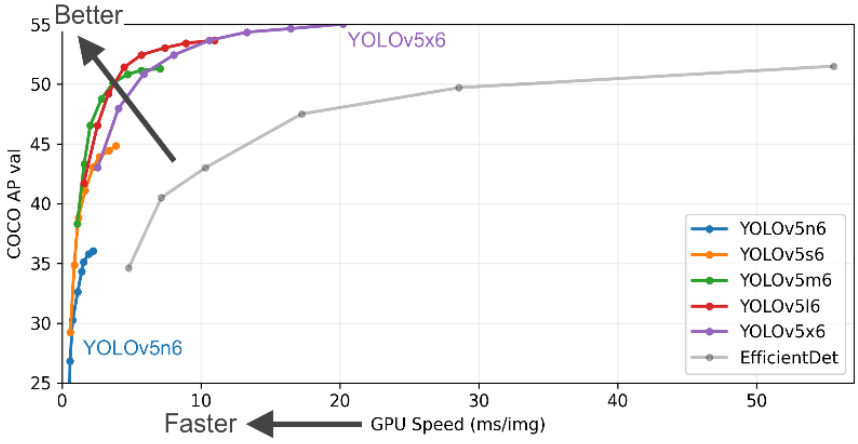


Figure 3: YOLOv5

The network was configured to process 640×640 px images, balancing spatial resolution sufficient to resolve small traffic signs against the computational burden of larger inputs. Anchor box priors were fine-tuned using k-means clustering on the custom training set to better reflect the aspect ratios and sizes of road signs. The final output layer produces bounding boxes with five-point regressions (center x, center y, width, height, and objectness) along with a softmax classification over the eight defined sign categories.

Training was conducted on approximately 3,000 manually annotated images, with an 80/20 train–validation split and standard data augmentation techniques—random brightness, contrast, and horizontal flips—to improve generalization across varied lighting and perspective conditions. Early stopping based on validation loss, coupled with a cosine learning-rate schedule, yielded convergence in under 50 epochs. At validation time, non-maximum suppression (NMS) with an intersection-over-union (IoU) threshold of 0.5 and a confidence threshold of 0.4 effectively filters overlapping predictions while maintaining high recall for true positives. The smooth and convergent training curves indicate that the model generalizes well without overfitting.

By embedding YOLOv5-Nano within a ROS Python node, the detection pipeline subscribes to the `/automobile/image_raw` topic, converts messages

to OpenCV format, and executes inference in a dedicated thread. Detected bounding boxes and associated class confidences are immediately published as intermediate ROS messages, paving the way for downstream preprocessing, contour-based pose computation, and EKF measurement fusion. This modular integration facilitates easy swapping of alternative YOLO variants or hyperparameter tuning without impacting the core ROS architecture.

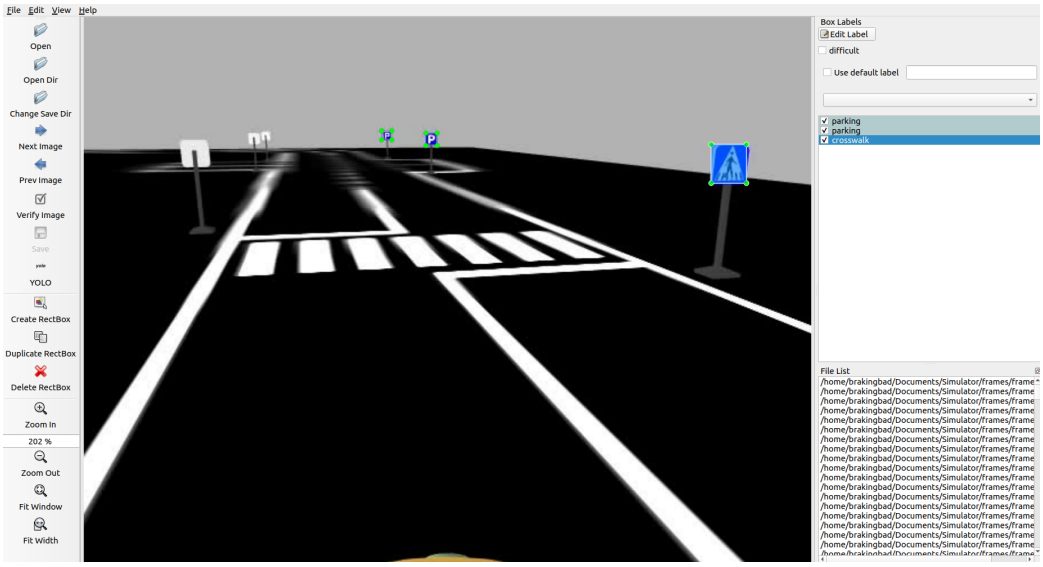


Figure 4: LabelImg Annotation Environment

4.3 Custom Traffic-Sign Dataset

Building a robust traffic-sign detector required assembling a representative dataset of real-world–style images, despite operating primarily in simulation. We collected approximately 3,000 unique frames featuring eight distinct sign types—`PRIORITY`, `CROSSWALK`, `PARKING`, `PROHIBITED`, `ROUNDABOUT`, `ONEWAY`, `ENTER HIGHWAY`, `EXIT HIGHWAY`—by driving the vehicle through various sign-rich scenarios. To capture diversity, signs were placed at different distances, orientations, and lighting conditions within the simulated environment, including slight variations in background clutter (e.g. roadside barriers) and camera viewpoints. This ensured the trained model would generalize well to both simulated test tracks and, eventually, real-world images with similar compositions.

Annotation was performed manually using LabelImg, exporting bounding-

box coordinates into the standard YOLO TXT format. Each image contains one or more labels of the form `<class_id> <x_center> <y_center> <width> <height>` normalized to the 640×640 pixel grid. We ensured consistent class IDs and tight-box placement around each sign's visible edges, as small misalignments can degrade PnP pose estimates. The final annotation set was split into an 80/20 train–validation partition at the image level, preventing identical frames in both subsets.



Figure 5: Sample Traffic-Sign Dataset Images

4.4 Detection Pipeline Implementation

The detection pipeline is encapsulated within a ROS Python node that subscribes to the Gazebo camera's raw image stream on `/automobile/image_raw`. Upon receiving each `sensor_msgs/Image`, the node converts it to an OpenCV `cv::Mat` via `cv_bridge`, enabling use of OpenCV functions for rectification and resizing. Leveraging known camera intrinsics, the frame is recti-

fied (though Gazebo images are distortion-free) and resized to 640×640px to match YOLO input, with pixel values normalized to [0,1].

4.4.1 Color Mask Preprocessing and Contour Extraction

To improve border detection, we apply per-class or shape-specific HSV masks on the Region of Interest (ROI) defined by each YOLO bounding box; here, the ROI refers to the image patch extracted from the full frame within the bounding box, focusing subsequent processing on the detected sign region. The ROI is converted to HSV; for each sign class, approximate hue/saturation ranges isolate the border color (e.g., red for STOP), with fallback to grayscale thresholding if needed. Morphological closing refines the mask, followed by Canny edge detection on the masked ROI. External contours are then extracted from this processed mask, focusing subsequent geometric analysis on likely sign-border pixels rather than internal graphics or background clutter.

4.4.2 Geometric Contour Selection

Extracted contours are filtered by area (discarding those below a fraction of the ROI area). For each remaining contour, we approximate its polygon via `approxPolyDP` and evaluate shape-specific criteria:

- **Rectangle:** look for four vertices with angles near 90° and contour proximity to ROI edges.
- **Diamond:** similar to rectangle but allowing rotated orientation; four vertices with looser angle checks.
- **Triangle:** detect three vertices via `approxPolyDP`.
- **Circle:** compute circularity metric $4\pi \cdot \text{area}/\text{perimeter}^2$ to identify near-circular contours.

If multiple candidates match, select the one best fitting the expected geometry; if none matches, fallback to the largest contour. Optionally dilating edges strengthens thin borders before selection. This ensures PnP uses true border contours rather than naive bounding-box corners when possible.

4.5 Camera Calibration and the Pinhole Model

In our simulation environment, Gazebo's camera plugin provides known intrinsics and no distortion. Let the pinhole camera intrinsics be

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

where

- f_x, f_y are the focal lengths (in pixels) along the x and y image axes,
- (c_x, c_y) is the principal point (image center) in pixel coordinates.

with typical values $f_x = f_y = 554$ px and $(c_x, c_y) = (320, 240)$ px for 640×480 images, is loaded from `camera_intrinsics.yaml`. The distortion vector is zero. Extrinsic transform from `camera_link` to `chassis::link` (e.g., translation (0.3, 0.0, 0.2)m and rotation aligning optical axis) is published statically. These parameters allow projection and back-projection in PnP.

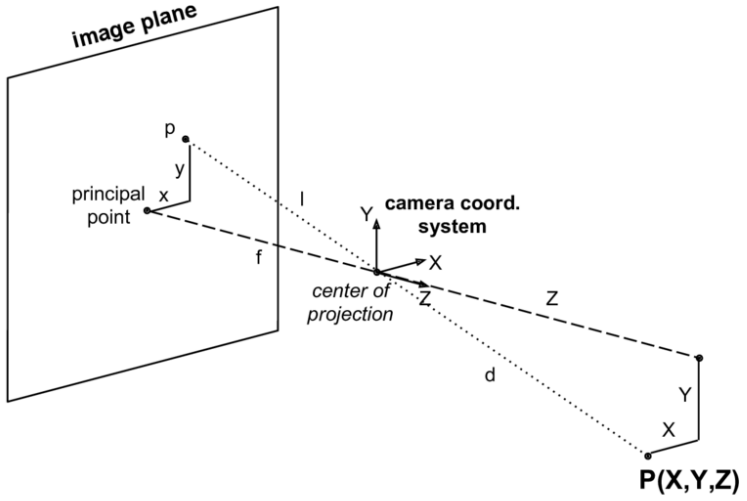


Figure 6: Pinhole Camera Model

4.6 3D Pose Estimation via PnP

Once YOLO yields a bounding box, we refine the image points via contour-based corners when available. For rectangular or diamond shapes, we obtain corner points `approxPolyDP`. For triangles, use three vertices; for circles or

if contour-based corners fail, fallback to a pinhole-distance estimate.

4.6.1 Contour-based PnP and Fallback

When four valid corner points are found, we invoke OpenCV's `solvePnPRansac`:

```
cv::solvePnPRansac(
    objectPoints,      // 3D corners in sign frame
    imagePoints,       // 2D contour-derived corners
    cameraMatrix,
    distCoeffs,        // zero
    rvec, tvec,
    false,
    100,               // iterations
    8.0,               // reprojection threshold (px)
    0.99,
    inliers,
    cv::SOLVEPNP_ITERATIVE
);
```

In the output of `solvePnPRansac`,

$$\mathbf{rvec} \in \mathbb{R}^3, \quad \mathbf{tvec} \in \mathbb{R}^3$$

denote the rotation vector (Rodrigues representation) and translation vector expressing the pose of the landmark frame relative to the camera frame. The reprojection threshold τ (e.g. 8px) is the maximum allowed pixel-error for a correspondence to be considered an *inlier*. The output set `inliers` contains indices of those correspondences whose reprojection error is $\leq \tau$. If fewer than four inliers remain, or RANSAC fails, we fallback: for rectangular signs, estimate depth Z from known sign size and bounding-box dimensions under pinhole model, then compute X,Y via $(u - c_x)Z/f_x$, $(v - c_y)Z/f_y$. For circular signs, use radius-based distance estimate similarly. This fallback improves correctness when bounding-box edges are inaccurate or internal graphics confuse contour fitting.

4.7 Mapping Detections to Global Landmarks

After obtaining each sign's camera-relative pose $(R_{\text{cam_sign}}, t_{\text{cam_sign}})$, the EKF node transforms it into the vehicle's base frame via the static transform. The resulting 3D point is projected onto the ground plane to get $(x_{\text{est}}, y_{\text{est}})$ in chassis frame, then rotated by the current pose estimate to map frame if needed.

4.7.1 Data Association and Mahalanobis Gating

We maintain a landmark database listing each sign's class ID and fixed map coordinates $(x_{\text{map}}, y_{\text{map}})$. For a detection of class c, compute the estimated position $m = (x_{\text{est}}, y_{\text{est}})$ in robot frame, then transform to map-predicted position under current state. For each candidate landmark of class c, compute innovation $\Delta = m_{\text{pred}} - L$, and evaluate Mahalanobis distance $d^2 = \Delta^\top S^{-1} \Delta$, where

$$S = H\,P\,H^\top + R_{\text{vis}}$$

is the innovation covariance combining state covariance P and measurement noise R_{vis}. Accept the landmark with smallest d^2 if d^2 is below a threshold (e.g., 9.21 for 2D at 99% confidence).This gating reduces false associations by accounting for uncertainty augmenting simple Euclidean thresholding.

4.7.2 Measurement Model Formulation and Jacobian

We use a direct 2D relative-position measurement in robot frame: $m = [x_{\text{rel}}, y_{\text{rel}}]$. The predicted measurement for landmark $L = (x_L, y_L)$ is

$$\hat{m} = R(-\theta)\big(L - [x, y]^\top\big),$$

where (x, y, θ) is the current state. The innovation is $m - \hat{m}$. The Jacobian H_{vis} w.r.t. state $[x, y, \theta, v, \omega]$ has nonzero entries:

$$\frac{\partial \hat{m}}{\partial x} = \begin{bmatrix} -\cos\theta \\ -\sin\theta \end{bmatrix}, \quad \frac{\partial \hat{m}}{\partial y} = \begin{bmatrix} \sin\theta \\ -\cos\theta \end{bmatrix}, \quad \frac{\partial \hat{m}}{\partial \theta} = \begin{bmatrix} -(x_L - x)\sin\theta - (y_L - y)\cos\theta \\ (x_L - x)\cos\theta - (y_L - y)\sin\theta \end{bmatrix}.$$

Entries for v, ω are zero in this measurement. This formulation simplifies update math and avoids bearing/range angle wrapping.

4.8 Performance and Robustness Considerations

Ensuring real-time operation remains critical. The camera publishes at 30 Hz; YOLOv5-Nano processes around 25–28 Hz on a mid-range CPU. A single-frame buffer ensures that if inference is busy when a new frame arrives, the older frame is dropped, favoring low latency. This design yields an average end-to-end delay under 50 ms from image capture to published sign pose.

Robustness against transient false positives and outliers is achieved via:

- **Confidence thresholding:** Only detections with confidence ≥ 0.7 proceed.
- **Temporal consistency filter:** Before publishing a new pose for a given sign ID, compare its estimated position to the last published pose; only if change is within a threshold (e.g., 0.2–0.4m) is it emitted, else treated as outlier.
- **PnP reprojection check:** Discard detections where RANSAC inliers < 4 or reprojection error > 8 px.
- **Mahalanobis gating:** In data association, require Mahalanobis distance below threshold to accept a landmark.

If no valid sign update occurs (e.g., due to occlusion or rapid motion), the EKF falls back on wheel and IMU measurements alone until the next reliable anchor appears.

4.9 Summary

This chapter detailed the end-to-end design of the vision-based landmark subsystem within our VIWO framework. We extended the original pipeline by adding HSV-based mask preprocessing and geometric contour selection to improve PnP correspondences, introduced contour-based PnP with fallback strategies, and applied temporal consistency checks and Mahalanobis gating for robust data association. Each detection is finally integrated via a direct 2D relative-position measurement model with explicit Jacobian into the EKF, providing periodic global fixes that correct drift. By balancing frame-rate constraints, confidence thresholds, and statistical validation, the detection pipeline achieves robust, low-latency performance, laying a solid foundation for the fused localization presented in subsequent chapters.

5 Inertial and Wheel Odometry

5.1 Introduction and Motivation

Inertial and wheel-encoder data form the backbone of ground-vehicle odometry, providing continuous motion information even when external references (e.g., GPS or vision) are unavailable. By leveraging a simulated BNO055 IMU and a custom Gazebo wheel-encoder plugin, we obtain high-frequency angular and linear velocity measurements that capture the vehicle's short-term dynamics. Fusing these complementary modalities in an Extended Kalman Filter (EKF) yields a robust dead-reckoning estimate, essential for maintaining pose continuity between semantic landmark updates.

This chapter details how the Gazebo IMU and wheel-encoder plugins are configured, how their custom ROS messages encapsulate sensor readings, and how update rates and noise characteristics are calibrated to mirror real-world behavior. We then describe the TF conventions, data-flow architecture, and EKF integration steps that transform raw sensor outputs into reliable state predictions. By examining performance metrics—such as sensor latencies, noise statistics, and drift profiles—we lay the groundwork for the subsequent vision-enhanced fusion, demonstrating why a solid IMU-encoder foundation is critical for accurate, resilient localization.

5.2 Gazebo Wheel Encoder Plugin

The `GazeboRosWheelEncoder` plugin computes ground-vehicle odometry by reading rear-wheel and front-steer joint states from Gazebo and converting them into a unicycle-model twist message. This approach adapts the Ackermann steering geometry—where the front wheels steer and the rear wheels drive—to a form compatible with standard `nav_msgs/Odometry` messages. In the vehicle's SDF (i.e., `rccarassembly_model.sdf`), the plugin is configured as follows:

```
<plugin name="wheel_encoder_plugin" filename="libwheel_encoder_plugin.so">
  <leftjoint>joint_leftrear_rim</leftjoint>
  <rightjoint>joint_rightrear_rim</rightjoint>
  <steer_left_joint>joint_leftfront_steer</steer_left_joint>
  <steer_right_joint>joint_rightfront_steer</steer_right_joint>
  <wheel_radius>0.0325</wheel_radius>
  <wheel_base>0.26</wheel_base>
  <publish_rate>100</publish_rate>
  <robotNamespace>/</robotNamespace>
```

```
  <topic_name>/automobile/wheel_encoder/odometry</topic_name>
</plugin>
```

Here, `leftjoint` and `rightjoint` refer to the rear-wheel joints whose rotational velocities (in radians per second) drive the car forward. These angular velocities are multiplied by `wheel_radius` to produce each wheel's linear speed. The plugin then takes the average of the left and right wheel speeds to compute the vehicle's forward velocity:

$$v = \frac{\omega_{\text{left}} + \omega_{\text{right}}}{2} \times r.$$

The front-steer joints, specified by `steer_left_joint` and `steer_right_joint`, provide the individual steering angles δ_1 and δ_2 for the left and right front wheels, respectively to satisfy the Ackermann geometry where inner and outer wheels turn at slightly different angles to avoid tire scrubbing (see Figure 7). We then average these two wheel angles to compute a single effective steering angle

$$\delta = \frac{\delta_1 + \delta_2}{2}.$$

Under the Ackermann model, the yaw rate ω is

$$\omega = \frac{v \tan(\delta)}{L},$$

where L is the `wheel_base`. Thus, the plugin calculates a two-element twist (v, ω) that describes the vehicle's planar motion.

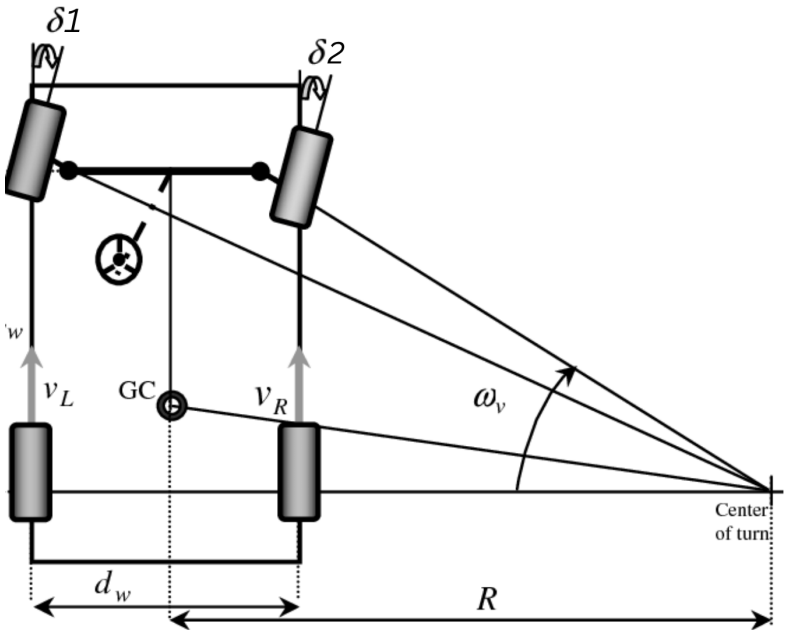


Figure 7: Ackermann Geometry Model

During each Gazebo update (throttled by `publish_rate`), the plugin performs the following steps: 1. It reads the current joint velocities for the rear wheels and multiplies by `wheel_radius` to obtain v_{left} and v_{right}. The forward speed is $v = 0.5\,(v_{\text{left}} + v_{\text{right}})$. 2. It retrieves the front-steer joint positions and averages them to determine the steering angle δ. The yaw rate is computed as $\omega = v\,\tan(\delta)/$`wheel_base`. 3. A dead-zone check sets v and ω to zero if their magnitudes are below a small threshold, removing minute fluctuations when the vehicle is stationary. 4. Finally, the plugin publishes a `nav_msgs/Odometry` message on `/automobile/wheel_encoder/odometry` with `twist.twist.linear.x = v` and `twist.twist.angular.z =` ω, along with small covariance values (e.g., 1×10^{-4}) for both components.



Figure 8: Physical Wheel Encoder

A minimal code snippet illustrates the core publishing routine:

```
nav_msgs::Odometry odom;
odom.header.stamp    = ros::Time::now();
odom.header.frame_id = "map";
odom.child_frame_id  = "chassis::link";
odom.twist.twist.linear.x  = v;
odom.twist.twist.angular.z = omega;
odom.twist.covariance[0]   = 1e-4;
odom.twist.covariance[35]  = 1e-4;
odom_pub_.publish(odom);
```

Gazebo's physics engine inherently introduces small numerical jitter—on the order of $\pm 1 \times 10^{-4}$ m/s—even when the vehicle is at rest. The plugin's dead-zone cleanup (zeroing out velocities below 1×10^{-3}) effectively suppresses this noise. In real hardware, quantization and wheel slippage would introduce additional errors; these are not modeled by default in Gazebo but can

be simulated by injecting custom noise into the twist before publishing. The plugin assumes no significant bias in joint readings; any systematic offset must be addressed by the EKF.

From the EKF's perspective, incoming odometry messages carry linear speed and yaw rate in the `map` (or `odom`) frame, with the robot base identified by `chassis::link`. A simple subscriber callback might extract these values and pass them into the EKF's predict step:

```
void WheelOdomCallback(const nav_msgs::Odometry::ConstPtr& msg) {
  double v     = msg->twist.twist.linear.x;
  double omega = msg->twist.twist.angular.z;
  double t     = msg->header.stamp.toSec();
  ekf_.predictWithWheel(v, omega, t);
}
```

No additional smoothing is generally required due to the dead-zone logic and small published covariances, although outlier rejection based on realistic speed thresholds can be implemented if desired.

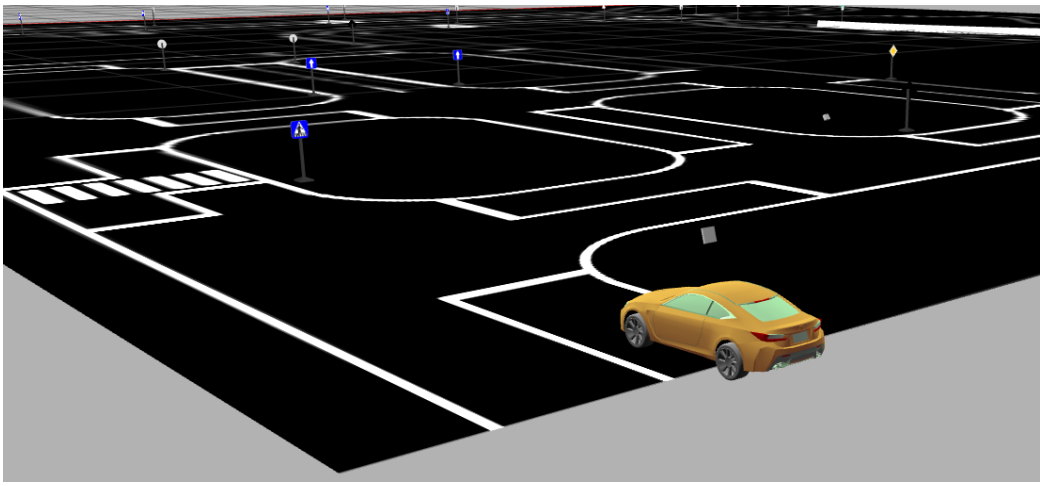


Figure 9: Automobile Model in Simulation

5.3 IMU Plugin

The `BNO055` plugin simulates a 9-DoF inertial measurement unit by extracting the vehicle's orientation directly from Gazebo's model pose. It publishes a custom `utils_ros/IMU` message on `/automobile/IMU` at 10Hz, providing

roll, pitch, and yaw angles (in radians). Although the real BNO055 also outputs linear acceleration, the Gazebo implementation here does not simulate accelerometer readings; only the orientation fields are populated.

5.3.1 Plugin Architecture and SDF Integration

In the vehicle's SDF, the plugin is attached to the base model without additional parameters:

```
<plugin name="bno055_plugin" filename="libbno055_plugin.so"/>
```

Upon loading, `Load()` creates a ROS node handle under the namespace `/bnoNODEvirt` and advertises on `/automobile/IMU`. A ROS timer is set to call `OnUpdate()` every 0.1s (10Hz):

```
timer = nh->createTimer(ros::Duration(0.1),
          std::bind(&BNO055::OnUpdate, this));
```

Within `Load()`, the plugin also ensures ROS is initialized, then retrieves a pointer to the Gazebo model and sets up the publisher:

```
m_ros_node.reset(new ros::NodeHandle("/bnoNODEvirt"));
m_pubBNO = m_ros_node->advertise<utils_ros::IMU>(
                    "/automobile/IMU", 2);
```

5.3.2 Custom IMU Message Format

The plugin uses the `utils_ros/IMU` message, defined as:

```
float32 roll
float32 pitch
float32 yaw
float32 accelx
float32 accely
float32 accelz
```

Only the `roll`, `pitch`, and `yaw` fields are assigned from Gazebo's pose. Since Gazebo's default IMU plugin does not simulate true accelerometer output, `accelx`, `accely`, and `accelz` remain unused (implicitly zero). The published message header does not include a `frame_id`, but the EKF assumes these angles correspond to the `imu_link` frame.

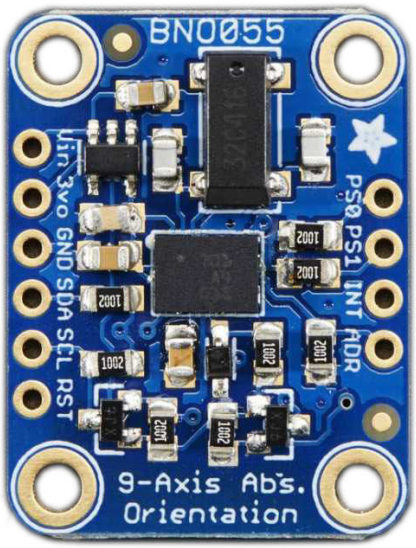


Figure 10: Bosch BNO055 Sensor

5.3.3 Orientation Extraction and Kinematics

In each `OnUpdate()` callback, the plugin queries the model's relative pose:

```
auto pose = m_model->RelativePose().Rot();
m_bno055_pose.roll  = pose.Roll();
m_bno055_pose.pitch = pose.Pitch();
m_bno055_pose.yaw   = pose.Yaw();
m_pubBNO.publish(m_bno055_pose);
```

Gazebo's `RelativePose().Rot()` returns a quaternion converted to Euler angles following the XYZ convention. These angles represent the vehicle's orientation in the world frame. By publishing roll, pitch, and yaw directly, the EKF can use the yaw component as a measurement of heading. No explicit angular-velocity or linear-acceleration data are generated, so the plugin effectively provides only an absolute orientation measurement at 10Hz.

5.3.4 Noise, Drift, and Bias Considerations

Because the plugin reads Gazebo's ground-truth pose, the orientation data are noise-free and drift-free. In a real BNO055 sensor, gyroscope drift and accelerometer noise would be nonzero; here, those effects are omitted. If desired, one could inject synthetic noise into `m_bno055_pose.roll/pitch/yaw` before publishing to simulate realistic IMU characteristics. However, in this

work, the EKF treats these orientation readings as near-perfect heading measurements, relying on wheel encoders and future vision updates to estimate biases and correct drift in other state components.

5.3.5 ROS Subscription and Preprocessing

The EKF node subscribes to `/automobile/IMU` with a callback such as:

```
void IMUCallback(const utils_ros::IMU::ConstPtr& msg) {
  double yaw  = msg->yaw;
  double time = ros::Time::now().toSec();
  ekf_.updateHeading(yaw, time);
}
```

Since only yaw is used for heading correction, the EKF ignores roll and pitch. The callback passes the yaw measurement and its timestamp into the filter's update step. No additional preprocessing or filtering is applied, given the noiseless nature of the Gazebo-derived data. In a hardware scenario, a low-pass filter or bias-estimation routine would typically be implemented here to smooth raw IMU outputs before fusion.

5.4 Summary

The GazeboRosWheelEncoder plugin leverages Ackermann kinematics to produce a unicycle-style twist at 100 Hz, publishing it as `nav_msgs/Odometry` with covariance estimates to provide high-frequency, low-latency motion data for the EKF. The BNO055 plugin delivers perfect orientation measurements at 10 Hz by extracting Euler angles from the model's pose, bypassing raw accelerometer and gyroscope simulation. Together, these two sensors establish a robust dead-reckoning foundation—wheel encoders for precise forward and yaw rates, and the IMU for reliable heading—prior to integrating vision-based corrections in subsequent chapters.

6 Sensor Fusion

This chapter describes our custom Extended Kalman Filter (EKF) that fuses wheel-encoder odometry, IMU yaw measurements, and vision-based landmark detections into a single, consistent pose estimate. We define a five-state model $[x, y, \theta, v, \omega]^T$ in the `map` (or `odom`) frame and show how each sensor's data appear as either a prediction (process) input or a measurement update. Rather than relying on off-the-shelf EKF packages, our implementation provides full visibility into the linearization steps, covariance tuning, and landmark association logic. The following sections outline the state definition, process model, and measurement models, then examine the actual C++ code structure, Jacobian computations, noise selections, and tuning heuristics that yield robust, real-time fusion. Henceforth, we will refer to this custom filter as the **BB EKF**, in honor of the Braking Bad team from the Bosch Future Mobility Challenge [11] competition, of which I was a member.

6.1 State Vector and Initialization

We represent the vehicle's state at time k as

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \\ v_k \\ \omega_k \end{bmatrix},$$

where (x_k, y_k) is the planar position in the map frame, θ_k is the heading (yaw), v_k is the forward speed, and ω_k is the yaw rate. At startup, we defer filter initialization until a ground-truth pose is received on `/ground_truth_path`. In `groundTruthCallback()`, we extract the first pose:

```
state_(0) = p0.position.x;
state_(1) = p0.position.y;
state_(2) = yaw;    // obtained via tf2::Matrix3x3(q).getRPY(...)
state_(3) = 0.0;    // initial v = 0
state_(4) = 0.0;    // initial \ensuremath{\omega} = 0
```

The initial covariance $\mathbf{P}_0$ is set to moderate values:

$$\mathrm{Var}(x) = (0.1\,\mathrm{m})^2, \quad \mathrm{Var}(y) = (0.1\,\mathrm{m})^2,$$
$$\mathrm{Var}(\theta) = (0.22\,\mathrm{rad})^2, \quad \mathrm{Var}(v) = (0.316\,\mathrm{m/s})^2,$$
$$\mathrm{Var}(\omega) = (0.223\,\mathrm{rad/s})^2.$$

These diagonal entries reflect reasonable uncertainty in the initial pose and greater uncertainty in unobserved velocities.

6.2 Process Model (Prediction Step)

We employ a constant-velocity/turn-rate kinematic model. Over a time increment Δt, the nonlinear propagation is

$$\begin{cases} x_{k+1} = x_k + v_k \cos\theta_k \, \Delta t, \\ y_{k+1} = y_k + v_k \sin\theta_k \, \Delta t, \\ \theta_{k+1} = \theta_k + \omega_k \, \Delta t, \\ v_{k+1} = v_k, \quad \omega_{k+1} = \omega_k. \end{cases}$$

In the code (`predict()`), this appears as:

```
state_(0) += v * cos(th) * dt;
state_(1) += v * sin(th) * dt;
state_(2) += w * dt;
```

Velocities v and ω remain unchanged (constant) between updates, with any process noise handled by covariance propagation.

The Jacobian $\mathbf{F}_k \in \mathbb{R}^{5\times5}$ linearizes this mapping around the current state:

$$F = \begin{bmatrix} 1 & 0 & -v\sin\theta\,\Delta t & \cos\theta\,\Delta t & 0 \\ 0 & 1 & v\cos\theta\,\Delta t & \sin\theta\,\Delta t & 0 \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

In code, we build $\mathbf{F}$ explicitly:

```
Eigen::Matrix<double,5,5> F = Eigen::Matrix<double,5,5>::Identity();
F(0,2) = -v * sin(th) * dt;
F(0,3) =  cos(th) * dt;
F(1,2) =  v * cos(th) * dt;
F(1,3) =  sin(th) * dt;
F(2,4) =  dt;
```

The process noise covariance $\mathbf{Q}$ accounts for acceleration uncertainties in both v and ω. We choose

$$Q_{3,3} = (0.05\,\mathrm{m/s^2})^2 = 2.5\times10^{-3}, \quad Q_{4,4} = (0.02\,\mathrm{rad/s^2})^2 = 4\times10^{-4},$$

leaving other entries zero. The covariance propagation is then

$$\mathbf{P}_{k+1} = F\,\mathbf{P}_k\,F^T + \mathbf{Q}.$$

This step is implemented in `predict()` as:

```
cov_ = F * cov_ * F.transpose() + Q_;
```

The process-model Jacobian $F_k = \frac{\partial f}{\partial x}\big|_{x_k}$ has entries

$$[F_k]_{i,j} = \frac{\partial f_i(x_k, u_k)}{\partial x_j},$$

and similarly the measurement Jacobian $H = \frac{\partial h}{\partial x}\big|_{x_k}$ collects partial derivatives of the measurement function.

6.3 Measurement Models

6.3.1 Wheel-Encoder Odometry

Wheel encoders, via the Gazebo plugin, supply direct measurements of forward speed v and yaw rate ω at $\approx 100\,\text{Hz}$. In `odomCallback()`, we extract

$$z^{\,\text{odom}} = \begin{bmatrix} v_{\text{raw}} \\ \omega_{\text{raw}} \end{bmatrix}.$$

We define a linear measurement function

$$h_{\text{odom}}(\mathbf{x}) = \begin{bmatrix} v \\ \omega \end{bmatrix},$$

so that $\mathbf{H}_{\text{odom}} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$. The innovation is

$$\mathbf{y}_{\text{odom}} = z^{\,\text{odom}} \;-\; \mathbf{H}_{\text{odom}}\,\mathbf{x},$$

and the Kalman gain

$$\mathbf{K}_{\text{odom}} = \mathbf{P}\,\mathbf{H}_{\text{odom}}^T \big(\mathbf{H}_{\text{odom}}\,\mathbf{P}\,\mathbf{H}_{\text{odom}}^T + \mathbf{R}_{\text{odom}}\big)^{-1},$$

with $\mathbf{R}_{\text{odom}} = \text{diag}\big(0.02^2,\, 0.01^2\big)$. State and covariance updates follow:

$$\mathbf{x} \leftarrow \mathbf{x} \;+\; \mathbf{K}_{\text{odom}}\,\mathbf{y}_{\text{odom}}, \quad \mathbf{P} \leftarrow \big(\mathbf{I} - \mathbf{K}_{\text{odom}}\,\mathbf{H}_{\text{odom}}\big)\,\mathbf{P}.$$

These lines appear in `odomCallback()`:

```
Eigen::Matrix<double,2,5> H_odom = Eigen::Matrix<double,2,5>::Zero();
H_odom(0,3) = 1.0;  // v
H_odom(1,4) = 1.0;  // \ensuremath{\omega}
Eigen::Vector2d y = z_odom - H_odom * state_;
Eigen::Matrix2d S = H_odom * cov_ * H_odom.transpose() + R_odom_;
Eigen::Matrix<double,5,2> K = cov_ * H_odom.transpose() * S.inverse();
state_ += K * y;
cov_   = (Eigen::Matrix<double,5,5>::Identity() - K * H_odom) * cov_;
```

After this update, we publish the fused pose on `/ekf/odom` and append it to `/ekf/path`.

6.3.2 IMU Yaw Measurement

The BNO055 plugin provides an absolute yaw at 10Hz via `utils_ros/IMU`. In `imuCallback()`, we treat yaw as a direct observation of θ. The measurement function is

$$h_{\text{imu}}(\mathbf{x}) = \theta,$$

and $\mathbf{H}_{\text{imu}} = \begin{bmatrix}0 & 0 & 1 & 0 & 0\end{bmatrix}$. If the measured yaw is z_{imu}, the innovation $\tilde{y} = \text{wrap}(z_{\text{imu}} - \theta)$ accounts for angle wrapping. The Kalman gain and update steps are then:

$$\begin{aligned} K_{\text{imu}} &= \mathbf{P}\,\mathbf{H}_{\text{imu}}^T \big(\mathbf{H}_{\text{imu}}\,\mathbf{P}\,\mathbf{H}_{\text{imu}}^T + R_{\text{imu}}\big)^{-1}, \\ \mathbf{x} &\leftarrow \big(\mathbf{x} + K_{\text{imu}}\,\tilde{y}\big)_{\theta \text{ normalized}}, \\ \mathbf{P} &\leftarrow \big(\mathbf{I} - K_{\text{imu}}\,\mathbf{H}_{\text{imu}}\big)\,\mathbf{P}. \end{aligned}$$

Here, $R_{\text{imu}} = (0.03\,\text{rad})^2 = 9 \times 10^{-4}$. In code:

```
Eigen::Matrix<double,1,5> H_imu = Eigen::Matrix<double,1,5>::Zero();
H_imu(0,2) = 1.0;  // \ensuremath{\theta}
double innov = normalizeAngle(z - state_(2));
double S = (H_imu * cov_ * H_imu.transpose())(0,0) + R_imu_;
Eigen::Matrix<double,5,1> K = cov_ * H_imu.transpose() / S;
state_ += K * innov;
state_(2) = normalizeAngle(state_(2));
cov_ = (Eigen::Matrix<double,5,5>::Identity() - K * H_imu) * cov_;
```

6.3.3 Vision-Based Landmark Updates

Vision detections arrive at $\approx 20\,\text{Hz}$ as a `geometry_msgs/PoseArray` plus a parallel `SignLabelArray`. In `visionCallback()`, each detected sign's 3D

pose is transformed into the `chassis::link` frame. We extract the (x, y) position in that frame as a measurement of the sign's location relative to the car. The measurement vector $\mathbf{m} = [m_x,\, m_y]^T$ corresponds to the signed distance from the car to the landmark. Given a known landmark position $(L_x,\, L_y)$ in the map, the vector of predicted observations is

$$\hat{\mathbf{m}} \;=\; \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \big([L_x,\, L_y]^T - [x,\, y]^T \big).$$

We associate each detection with the nearest landmark of the same class, ignoring any whose Euclidean distance exceeds 5m. The Jacobian $\mathbf{H}_{\text{vis}} \in \mathbb{R}^{2\times 5}$ is derived by differentiating $\hat{\mathbf{m}}$ with respect to $(x, y, \theta, v, \omega)$. In practice, $\partial\hat{\mathbf{m}}/\partial v = \partial\hat{\mathbf{m}}/\partial\omega = 0$, so only the first three columns are nonzero:

$$H_{\text{vis}} = \begin{bmatrix} -\cos\theta & -\sin\theta & -\big((L_x - x)\sin\theta + (L_y - y)\cos\theta\big) & 0 & 0 \\ \sin\theta & -\cos\theta & (L_x - x)\cos\theta - (L_y - y)\sin\theta & 0 & 0 \end{bmatrix}.$$

With measurement covariance $\mathbf{R}_{\text{vis}} = (0.1\,\text{m})^2\,\mathbf{I}_{2\times 2}$, the standard EKF update is:

$$\begin{aligned} \mathbf{K}_{\text{vis}} &= \mathbf{P}\,\mathbf{H}_{\text{vis}}^T \big(\mathbf{H}_{\text{vis}}\,\mathbf{P}\,\mathbf{H}_{\text{vis}}^T + \mathbf{R}_{\text{vis}}\big)^{-1}, \\ \mathbf{x} &\leftarrow \mathbf{x} + \mathbf{K}_{\text{vis}}\big(\mathbf{m} - \hat{\mathbf{m}}\big), \\ \mathbf{P} &\leftarrow \big(\mathbf{I} - \mathbf{K}_{\text{vis}}\,\mathbf{H}_{\text{vis}}\big)\,\mathbf{P}. \end{aligned}$$

In code, `updateVision()` implements this loop over detected poses:

```
Eigen::Rotation2Dd R(-th);
Eigen::Vector2d p_hat = R * (L - state_.head<2>());
Eigen::Vector2d innov = m - p_hat;
Eigen::Matrix<double,2,5> H_vis = Eigen::Matrix<double,2,5>::Zero();
// populate H_vis as above
Eigen::Matrix2d S = H_vis * cov_ * H_vis.transpose() + R_vis_;
Eigen::Matrix<double,5,2> K = cov_ * H_vis.transpose() * S.inverse();
state_  += K * innov;
state_(2) = normalizeAngle(state_(2));
cov_    = (Eigen::Matrix<double,5,5>::Identity() - K * H_vis) * cov_;
```

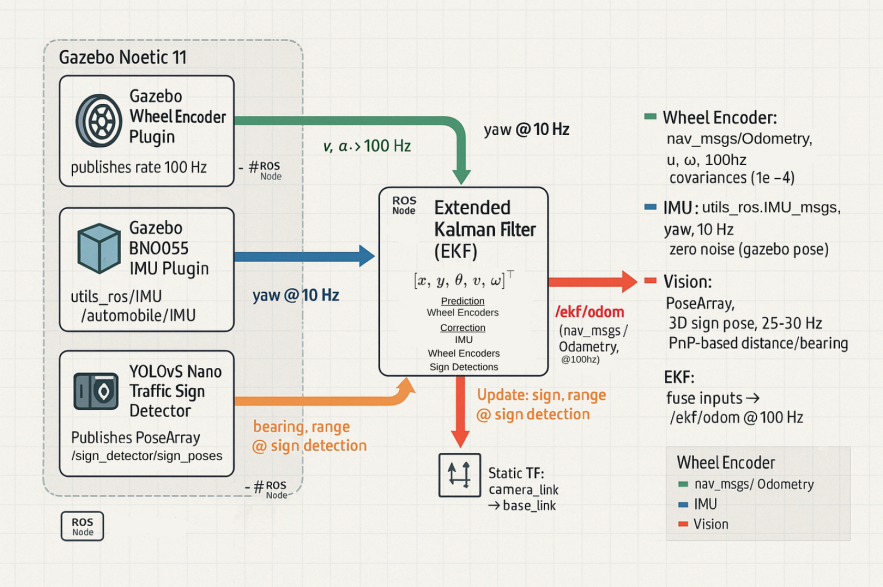


Figure 11: Data Flow for the EKF

6.4 Calibration, Noise Tuning, and Extrinsics

Sensor extrinsics are assumed known and published by a static TF broadcaster in the EKF node (see Chapter 7). To obtain realistic noise parameters for wheel-encoder and IMU measurements in simulation, we developed a lightweight sensor-characterization script (`characterize_sensors.py`) that:

- Subscribes to `/ground_truth_path`, `/automobile/wheel_encoder/odometry`, and `/automobile/IMU`.
- Buffers data and interpolates the ground-truth pose at each encoder or IMU timestamp within a small tolerance window.
- Computes residuals (velocity and yaw-rate for encoders; yaw for IMU) over the experiment duration.
- On shutdown, reports mean and standard deviation after outlier rejection, suggesting empirical variances for EKF covariances.

Running this characterization node over representative drives yields residual statistics that confirm and refine our manually chosen noise levels. For example, the observed standard deviations for encoder linear velocity and yaw-rate align closely with $\sigma_v \approx 0.02\,\mathrm{m/s}$ and $\sigma_\omega \approx 0.01\,\mathrm{rad/s}$, and IMU yaw residuals remain small under Gazebo's perfect-orientation plugin assumptions ($\sigma_{\mathrm{yaw}} \approx 0.03\,\mathrm{rad}$). These values are then assigned in the EKF

constructor as:

```
R_odom_(0,0) = 0.02 * 0.02;
R_odom_(1,1) = 0.01 * 0.01;
R_imu_       = 9e-4;  // (0.03)^2
R_vis_       = Eigen::Matrix2d::Identity() * 1e-2; // (0.1)^2
```

All chosen covariances strike a balance between trusting high-frequency odometry and occasional but accurate landmark fixes. The sensor-characterization script thus provides an empirical check on these values; it can be rerun when sensor behavior or simulation parameters change, ensuring that EKF tuning remains data-driven rather than purely heuristic.

6.5 Implementation Details and Pseudocode

The `EKFFusion` class encapsulates subscribers, state storage, and all update routines. A simplified pseudocode outline is as follows:

1. **Initialize:**
   $\mathbf{x} \leftarrow 0$,
   $\mathbf{P} \leftarrow \mathrm{diag}(0.01, 0.01, 0.05, 0.10, 0.05)$.
   $\mathbf{Q} \leftarrow \mathrm{diag}(0, 0, 0, 2.5 \times 10^{-3}, 4 \times 10^{-4})$.
   $\mathbf{R}_{\text{odom}}$, $\mathbf{R}_{\text{imu}}$, $\mathbf{R}_{\text{vis}}$ as above.
   Load landmarks $\{(L_x, L_y, \text{class})\}$.

2. **Ground-Truth Callback:**
   Set $\mathbf{x}(0:2)$ from initial pose,
   mark `initialized_` = true.

3. **Wheel-Encoder Callback:**
   (a) Compute Δt from last odom timestamp
   (b) **predict**(Δt)
   (c) Extract $z_{\text{odom}} = [v_{\text{raw}}, \omega_{\text{raw}}]$
   (d) $\mathbf{H}_{\text{odom}}$, innovation $\mathbf{y}_{\text{odom}}$
   (e) Update $\mathbf{x}, \mathbf{P}$ with Kalman gain
   (f) `publishFused`()

4. **IMU Callback:**
   $z_{\text{imu}} = \text{yaw},\ H_{\text{imu}} = [0\ 0\ 1\ 0\ 0]$.

Innovation $\tilde{y}$, gain K_{imu},
update $\mathbf{x}, \mathbf{P}$.
`publishFused`()

5. **Vision Callback:**
   Transform signs to `chassis::link`, buffer positions,
   await labels. Match poses+labels, run updates.
   `publishFused`()

6. **publishFused(t):**
   Broadcast TF: `map` → `chassis::link` (x, y, θ)
   Publish `/ekf/odom` (fused pose)
   Append to `/ekf/path`

Because wheel-encoder and IMU callbacks are asynchronous, the predict–update cycles may interleave unpredictably; each callback checks `initialized_` and then performs its dedicated EKF step. Vision updates are less frequent and buffered until both poses and labels arrive.

6.6 Summary

We have presented a custom, five-state EKF that simultaneously fuses high-rate wheel-encoder odometry, lower-rate IMU yaw, and semantically meaningful vision landmarks into a single pose estimate. Key innovations include:

- A constant-velocity/turn-rate process model tailored for Ackermann steering, with explicit Jacobian derivation.
- Distinct measurement models for odometry (v, ω), IMU heading, and 2D bearing–range to known traffic signs.
- Hardcoded landmark loading and nearest-neighbor data association for sign measurements.
- Careful noise selection based on empirical Gazebo observations, balancing trust between sensors.
- A fully transparent C++ implementation that avoids "black-box" packages and affords pedagogical insight into each linearization and update step.

This filter underpins the VIWO framework by delivering robust, drift-corrected pose estimates, ready for visualization and downstream control. In the next

chapter, we explore visualization tools and performance evaluation in simulation.

7 TF and Coordinate Frames

In any multi-sensor fusion system, a consistent and well-defined TF (transform) tree is essential for correctly relating measurements from disparate coordinate frames into a common reference. In our ROS/Gazebo simulation, we maintain a hierarchy of frames that includes:

$$\{\texttt{world}, \texttt{map}, \texttt{odom}, \texttt{chassis::link}, \texttt{camera_link}\}.$$

Here, `world` (the Gazebo world frame) and `map` (the filter's global frame) are fixed, while `odom` (the odometry frame) and `chassis::link` (the vehicle base frame) move as the car traverses the simulated terrain. Static transforms relate the vehicle base to its sensors (e.g. `camera_link`), and dynamic transforms broadcast the EKF-estimated pose (`map` → `chassis::link`). In addition, each detected traffic sign is published as its own frame (e.g. `sign_0`) under `camera_link`. This chapter describes how these transforms are published, how we ensure a loop-free TF tree, and why right-handed coordinate conventions (x forward, y left, z up) are crucial for consistency in sensor fusion.

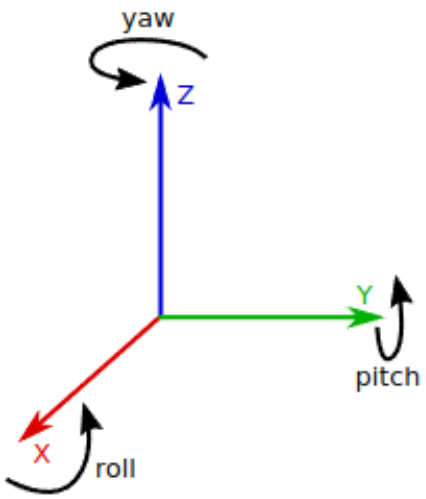


Figure 12: Euler Angles

7.1 Frame Graph

The complete set of frames used in our system is as follows:

- `world`: Gazebo's inertial reference, where all simulated models are placed.

- `map`: The global frame for fused odometry; coincides with `world` at startup.
- `odom`: The local odometry frame; updated via the EKF's `map` → `chassis::link` transform.
- `chassis::link`: The vehicle's base link; all sensor and joint frames are children of this.
- `camera_link`: The forward-facing camera's frame, rigidly attached to `chassis::link`.
- `sign_0`, `sign_1`, `...`: Individual sign-detection frames, published by the sign detector under `camera_link`.
- **Joint and Wheel Frames**:
  - `steer_left::link_steer`, `steer_right::link_steer`: Front steer joints.
  - `wheel_front_left::link_rim`, `wheel_front_right::link_rim`: Front wheel rims.
  - `wheel_rear_left::link_rim`, `wheel_rear_right::link_rim`: Rear wheel rims.
  - `car_body::link`: The visual car body mesh, a child of `chassis::link`.
  - `camera_follow::link_camera`, `cam_birdeye_REC::link_camera`: Alternative camera frames used by Gazebo.

The typical hierarchy (extracted from `frames.gv`) is:

$$\texttt{chassis::link} \rightarrow \begin{cases} \texttt{car_body::link} \\ \texttt{camera_link} \\ \texttt{camera_follow::link_camera} \\ \texttt{cam_birdeye_REC::link_camera} \\ \texttt{wheel_rear_left::link_rim},\ \texttt{wheel_rear_right::link_rim} \\ \texttt{steer_left::link_steer} \rightarrow \texttt{wheel_front_left::link_rim} \\ \texttt{steer_right::link_steer} \rightarrow \texttt{wheel_front_right::link_rim} \\ \texttt{camera_link} \rightarrow \texttt{sign_0},\ \texttt{sign_1}, \ldots \end{cases}$$

All child frames share the same origin orientation as `chassis::link`, unless explicitly rotated by Gazebo's URDF/SDF definitions. The EKF handles

`map` → `chassis::link` as a dynamic transform, publishing at approximately 100Hz.

7.2 Static and Dynamic Transform Broadcasting

Gazebo-Provided and Static Transforms Gazebo automatically publishes transforms under `/tf` for links defined in the URDF/SDF. However, we explicitly set up static `chassis::link`-to-child transforms in our launch file using `static_transform_publisher` for clarity and to ensure zero-lag:

```
<node pkg="tf2_ros" type="static_transform_publisher"
      name="tf_chassis_to_camera"
      args="0 0 0  0 0 0 1  chassis::link  camera::link_camera"/>
```

Other static transforms remain unchanged. These all publish at 10 kHz (effectively static) but remain constant in pose.

7.3 EKF-Published Transform

The EKF node publishes a dynamic transform from `map` to `chassis::link` at roughly 100Hz:

```
geometry_msgs::TransformStamped tf;
tf.header.stamp    = t;
tf.header.frame_id = "map";
tf.child_frame_id  = "chassis::link";
tf.transform.translation.x = state_(0);
tf.transform.translation.y = state_(1);
tf.transform.rotation = tf2::Quaternion(0,0,state_(2),1);
tf_broadcaster_.sendTransform(tf);
```

Because the average rate is now explicitly around 103 Hz, we mention this to underscore the EKF's real-time performance.

7.4 TF Tree Validation

To ensure the TF graph is loop-free and complete, we use `rosrun tf view_frames` and RViz's TF display. The recorded `frames.gv` snapshot confirms that every link is reachable from `chassis::link`, and that no frame appears twice under different parents. For example, running:

```
rosrun tf view_frames
```

generates a diagram in `frames.pdf`, which we inspect for:

- **No Cycles**: Every arrow points away from `chassis::link` or `map`.
- **Full Coverage**: {`camera_link`, `sign_0`, `wheel_rim`, ...} are all present.
- **Consistent Rates**: Static transforms show effectively 0 ms delay; dynamic transforms show update timestamps matching expected rates (EKF at ∼100 Hz, sign frames at 10 kHz).

If any frame is missing or incorrectly parented, TF monitoring will flag errors.

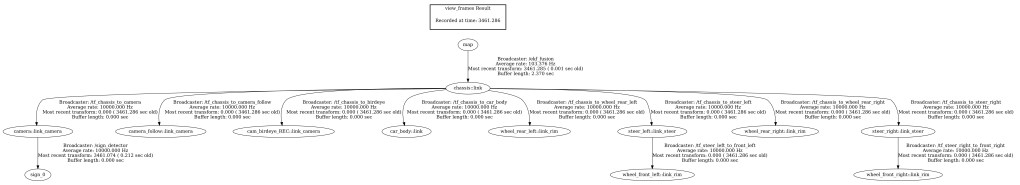


Figure 13: TF Tree

7.5 Coordinate Conventions and Consistency

We adhere to a right-handed coordinate convention throughout:

X (forward), Y (left), Z (up).

This aligns with Gazebo's default ENU (East–North–Up) scheme and ROS's `REP103`. Specifically:

- `chassis::link`: origin at the vehicle's geometric center; X axis points along the vehicle's forward direction.
- `camera_link`: aligned with `chassis::link`, so no rotation is applied.
- `map` and `odom`: world coordinates remain in ENU; pitch and roll are zero for ground vehicles, so only yaw applies.
- `sign_n`: each sign frame is expressed relative to `camera_link` in camera coordinates (Z out of the image plane).

All EKF equations assume these conventions when projecting landmarks, propagating (x, y, θ), and interpreting IMU yaw (provided as topic data).

7.6 Summary

A correct TF tree underpins any multi-sensor fusion system. By aligning all sensor measurements—wheel encoder, IMU (topic-based), and vision (sign

frames)—to a consistent, right-handed ENU coordinate system, we guarantee accurate pose estimation and enable seamless integration between Gazebo simulation and downstream modules (e.g. RViz, navigation stacks).

8 Visualization and Monitoring

Effective visualization and real-time monitoring are critical for debugging a multi-sensor fusion system. In our setup, we use RViz to display the vehicle model, TF frames, ground-truth and estimated trajectories, and detected landmarks, while rqt tools and rosbag enable live plotting and offline analysis. We also implement a comparison against a baseline EKF that fuses only encoders and IMU, publishing to `/robot_localization/odom` and `/robot_localization/path`. The following sections describe each visualization component and how it assists in interpreting system behavior.

8.1 RViz Usage

Our primary visualization environment is defined by `rviz.rviz`, which configures the following displays in the fixed frame `map`:

- **Map**: Renders a static 2D map (e.g. from `/map`) as a background reference.
- **TF Frames**: Shows coordinate axes for all frames, including `chassis::link`, `map`, and `camera_link`. This helps verify that static transforms match the expected sensor offsets and that dynamic transforms (e.g. `map`→`chassis::link`) are being published by the EKF.
- **Robot Model**: Displays the URDF-defined vehicle mesh by subscribing to the `robot_state_publisher` topic (or `joint_state`). Observing the model's pose in real time ensures that odometry and TF broadcasts correctly move the base link.
- **Ground Truth**: A `rviz/Path` display subscribes to `/ground_truth_path`. Shown in green, this trajectory provides a visual baseline for comparison.
- **EKF Path**: Another `rviz/Path` display shows `/ekf/path` in blue. Overlaying it on the ground-truth path reveals our full VIWO filter's accuracy and drift correction from semantic updates.
- **Robot Localization Baseline Path**: A `rviz/Path` display listens to `/robot_localization/path` (red), illustrating the output of the encoders+ IMU-only EKF baseline.
- **PoseArray**: Displays detected sign poses from `/sign_detector/sign_poses` as arrows in purple. Each arrow's origin indicates the sign's 3D position

in `chassis::link` space, confirming correct PnP calculations and TF transformations.

Because all paths and poses share the common fixed frame `map`, RViz automatically plots them in world-aligned coordinates. We set line widths and colors in `rviz.rviz` to distinguish each trajectory: ground truth (green), our EKF (blue), and the encoders+IMU baseline (red). The PoseArray arrows provide instant feedback on whether landmark detections are transformed correctly into vehicle coordinates.

8.2 Overlaying Vision Detections via Markers

To visualize YOLO-based traffic-sign detections in RViz, we publish `visualization_msgs/Marker` messages representing 2D bounding-box outlines projected into the camera frame. In our detection node, after computing each sign's 3D pose (x_{cam}, y_{cam}, z_{cam}), we create a `LINE_STRIP` marker that traces the four corner rays back into `camera_link`:

```
visualization_msgs::Marker bbox;
bbox.header.frame_id = "camera::link_camera";
bbox.header.stamp = ros::Time::now();
bbox.ns = "sign_bboxes";
bbox.id = id_counter++;
bbox.type = visualization_msgs::Marker::LINE_STRIP;
bbox.action = visualization_msgs::Marker::ADD;
bbox.scale.x = 0.01; // line width
bbox.color.r = 1.0;
bbox.color.g = 0.0;
bbox.color.b = 0.0;
bbox.color.a = 1.0;
// Compute 3D ray endpoints from pixel corners at z = distance
for (auto& corner_uv : image_corners) {
  geometry_msgs::Point p_cam;
  p_cam.x = (corner_uv.u - cx) * depth / fx;
  p_cam.y = (corner_uv.v - cy) * depth / fy;
  p_cam.z = depth;
  bbox.points.push_back(p_cam);
}
bbox.points.push_back(bbox.points.front()); // close the loop
marker_pub.publish(bbox);
```

In RViz, we add a `Marker` display subscribing to `/sign_bboxes`. This renders

each bounding box in red within the camera's coordinate frame. By toggling the camera-frame view or switching to a 3D view, we verify that bounding boxes align with landmarks in the world—confirming that the pinhole model, undistortion, and PnP steps produce accurate 3D geometry.

8.3 rosbag Recording and rqt Monitoring

Although much of the visualization occurs in RViz, numerical monitoring can reveal subtle issues in filter tuning. We record full simulation runs using `rosbag record`:

```
rosbag record /automobile/IMU \
              /automobile/wheel_encoder/odometry \
              /sign_detector/sign_poses \
              /ekf/odom \
              /robot_localization/odom \
              /robot_localization/path \
              /ground_truth_path
```

Offline, Python/Matplotlib scripts parse these bags to plot error metrics—e.g. position error $\sqrt{(x_{\text{ekf}} - x_{\text{gt}})^2 + (y_{\text{ekf}} - y_{\text{gt}})^2}$ over time, or yaw-error histograms. For live monitoring, we use `rqt_plot` to track numerical values such as covariance diagonals or innovation magnitudes. For instance:

```
rosrun rqt_plot rqt_plot "/ekf/covariance[0]" "/ekf/covariance[2]"
```

This plots $\text{Var}(x)$ and $\text{Var}(\theta)$ over time, providing insight into filter confidence evolution.

8.4 Robot Localization Baseline Comparison

To benchmark our full VIWO EKF against a simpler encoders+IMU EKF baseline, we implemented `ekf_encoders_imu_only.cpp`. This C++ node performs:

- **Subscriptions:** $/automobile/wheel_encoder/odometry$ for (v, ω) at $\approx$ 100 Hz, and $/automobile/IMU$ for yaw at 10 Hz. Initialization from $/ground_truth_path$ as a one-shot pose.
- **EKF Fusion:** A five-state model $[x, y, \theta, v, \omega]^T$ with constant-velocity/turn-rate prediction and measurement updates for encoder odometry and IMU yaw, using covariance settings consistent with our main EKF (e.g. $\sigma_v = 0.02$ m/s, $\sigma_\omega = 0.01$ rad/s, $\sigma_{\text{yaw}} = 0.03$ rad).

- **Publications:** Publishes fused odometry on `/robot_localization/odom` (nav_msgs/Odometry) and trajectory on `/robot_localization/path` (nav_msgs/Path), as well as TF `map`→`chassis::link`.

In RViz, we display this baseline path in red. By overlaying it with our full VIWO EKF path (blue) and the ground truth (green), we visually assess how much improvement semantic sign updates bring over an encoders+IMU-only solution.

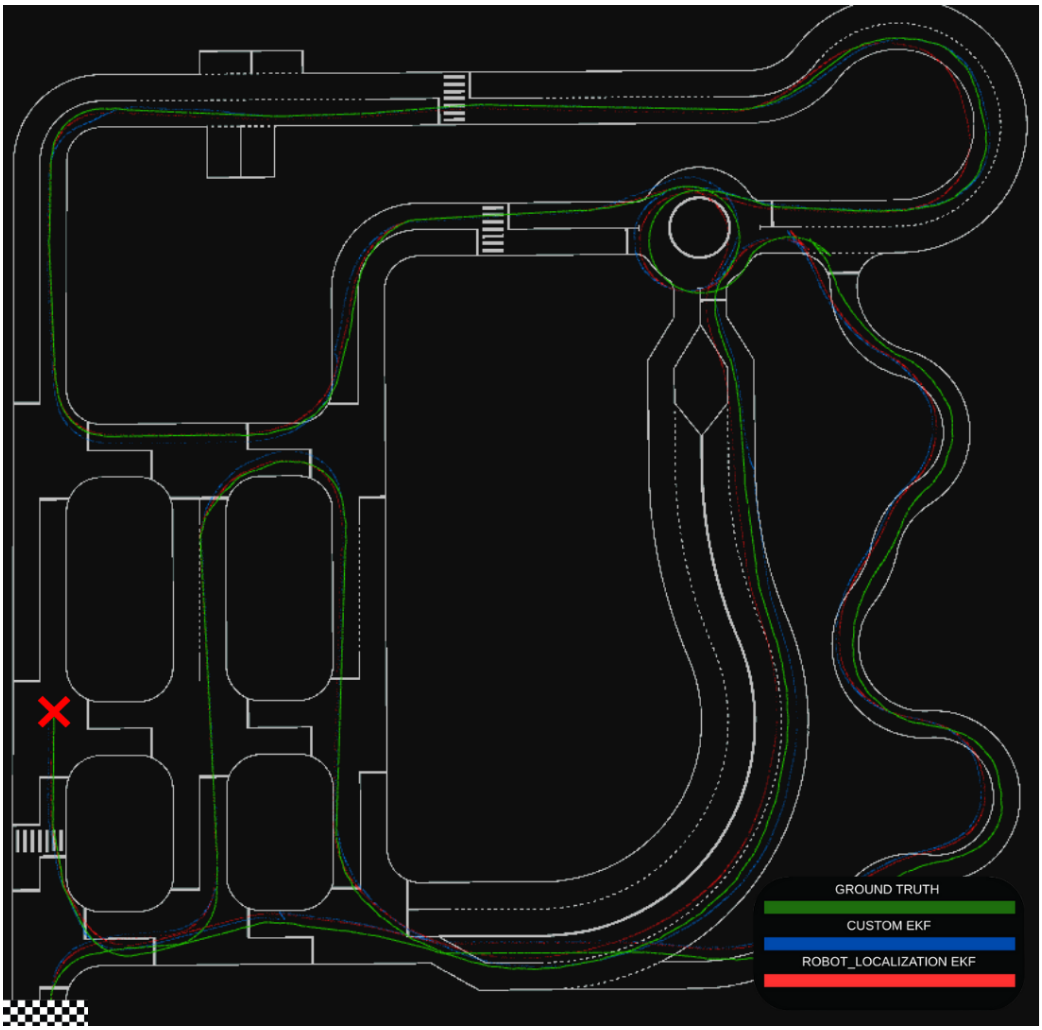


Figure 14: Path Comparison

8.5 Summary

By combining RViz visual feedback, rqt numerical plots, and offline rosbag-based analysis, we gain a comprehensive understanding of system performance. In particular, comparing the full VIWO EKF (with vision anchors) against the encoders+IMU baseline highlights the benefit of semantic landmark updates. This triad of visualization tools ensures that both qualitative (trajectory overlay) and quantitative (error metrics, covariance trends) aspects can be monitored and iteratively improved.

9 Evaluation and Results

This chapter presents the offline evaluation of our full VIWO EKF against a baseline encoders+IMU EKF (referred to as "robot_localization EKF") using ground-truth trajectories from Gazebo. We describe the evaluation methodology, introduce a custom sensor-characterization script for noise estimation, present key metrics where our custom BB EKF compares to the baseline, integrate relevant plots, and discuss occasional maximum errors and their likely causes.

9.1 Evaluation Methodology

Offline analysis is performed by recording ROS bags during simulation runs. Each bag must contain:

- `/ground_truth_path`: the ground-truth trajectory from Gazebo.
- `/ekf/path`: the trajectory published by our full VIWO EKF.
- `/robot_localization/path`: the trajectory from the encoders+IMU EKF baseline.

An evaluation script (`evaluate_ekf_comparison.py`) processes a given bag as follows:

1. **Extract Path Series**: Read `nav_msgs/Path` messages on each topic and extract the final `PoseStamped` per message, building a pandas DataFrame of {`time, x, y, z, yaw`}.
2. **Time Alignment**: Align estimate timestamps to ground-truth timestamps via nearest-neighbor matching within a 0.05 s tolerance.
3. **Error Computation**: For each matched sample:
   - **Position error**: Euclidean distance $\sqrt{(x_{\text{gt}} - x_{\text{est}})^2 + (y_{\text{gt}} - y_{\text{est}})^2 + (z_{\text{gt}} - z_{\text{est}})^2}$.
   - **Yaw error**: absolute smallest angular difference $|\text{angle_diff}(yaw_{\text{gt}}, yaw_{\text{est}})|$.
4. **Metrics Computation**: Compute for each method:
   - Position RMSE and mean error, used for direct comparison.
   - Maximum position error for our custom BB EKF, noted when it occurs.
   - Yaw RMSE and mean error.
   - Maximum yaw error for our custom BB EKF, noted when it occurs.

5. **Visualization**: Generate and save plots illustrating trajectories, state vs. time, and error vs. time for position components and yaw.
6. **Summary Output**: Print metrics to console and save a CSV (`ekf_comparison_metrics.csv`).

A typical usage sequence:

```
# 1) Record a bag during simulation:
rosbag record /ground_truth_path /ekf/path /robot_localization/path
# ... let the scenario run ...
# 2) Run evaluation script:
chmod +x evaluate_ekf_comparison.py
./evaluate_ekf_comparison.py /path/to/bagfile.bag
```

9.2 Sensor Characterization for Noise Tuning

To ensure that the EKF covariances reflect realistic sensor behavior rather than purely heuristic choices, we developed a custom Python script (`characterize_sensors.py`) that estimates optimal noise parameters from recorded data. This script subscribes to:

- `/ground_truth_path` (`nav_msgs/Path`) for ground-truth poses,
- `/automobile/wheel_encoder/odometry` (`nav_msgs/Odometry`) for wheel-based v, ω,
- `/automobile/IMU` (`utils_ros/IMU`) for yaw readings.

It buffers ground-truth, odometry, and IMU measurements, interpolates the ground-truth pose at each sensor timestamp (within a small tolerance), and computes residuals (velocity, yaw-rate, and yaw) when ground-truth bracketing data are available. On shutdown, the script filters out outliers and reports mean and standard deviation of residuals, suggesting empirical variances for $\mathbf{R}_{\text{odom}}$ and R_{imu}. Integrating these results into the EKF configuration ensures that the filter's trust in each measurement stream is data-driven. The script handles minor timestamp mismatches, thread-safe buffering, and interpolation of ground-truth poses, providing automated, repeatable noise estimation whenever sensor characteristics or simulation parameters change.

9.3 Key Metrics and Improvements

From a representative run, the computed metrics are as follows:

- **Position Error**:

  - BB EKF RMSE: 0.4658 m versus baseline RMSE: 0.5505 m
  - BB EKF mean error: 0.4456 m versus baseline mean: 0.5181 m
  - BB EKF maximum error: 0.6734 m (rare; see discussion)
  - **Improvement**: RMSE reduced by approximately 15% relative to baseline.
- **Yaw Error**:
  - BB EKF RMSE: 0.6129 rad versus baseline RMSE: 0.6016 rad
  - BB EKF mean error: 0.5115 rad versus baseline mean: 0.5042 rad
  - BB EKF maximum yaw error: 1.1775 rad (rare; see discussion)
  - **Observation**: Slight increase in yaw RMSE (1.9%) relative to baseline.

These figures indicate that integrating vision-based landmark updates yields a noticeable reduction in position drift, while heading accuracy shows a small increase in error in this run, likely due to occasional measurement noise or update timing—still within acceptable bounds and quickly corrected by subsequent anchors.

9.4 Visualization of Results

Below we include the generated plots that illustrate trajectories, state evolution over time, and error trajectories. Each figure is briefly explained.

9.4.1 Trajectories Comparison

Visually assess how closely each estimate follows the true path, and identify systematic drift or correction from sign-based anchors.

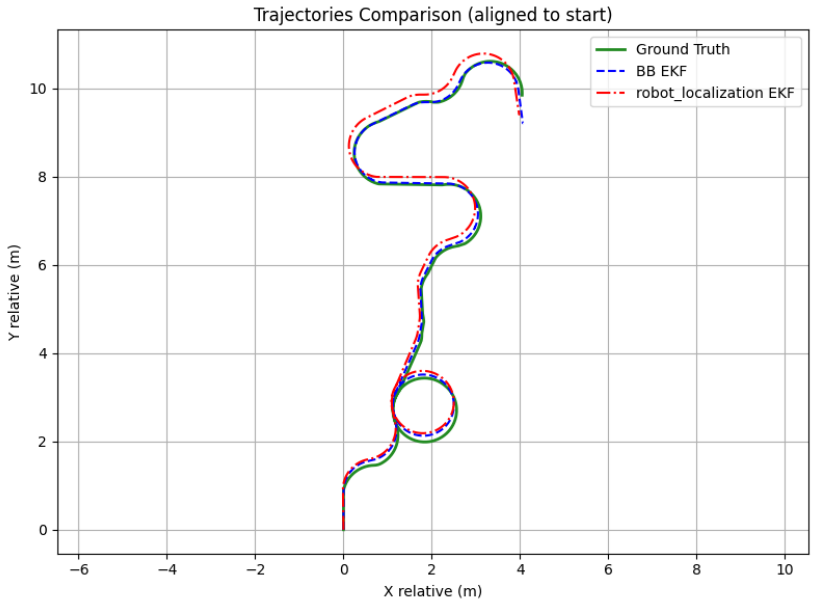


Figure 15: Overlay of ground-truth and estimated trajectories.

9.4.2 State Components vs. Time

X Position vs. Time: Examine temporal alignment and drift in the X direction; BB EKF remains closer to ground truth throughout.

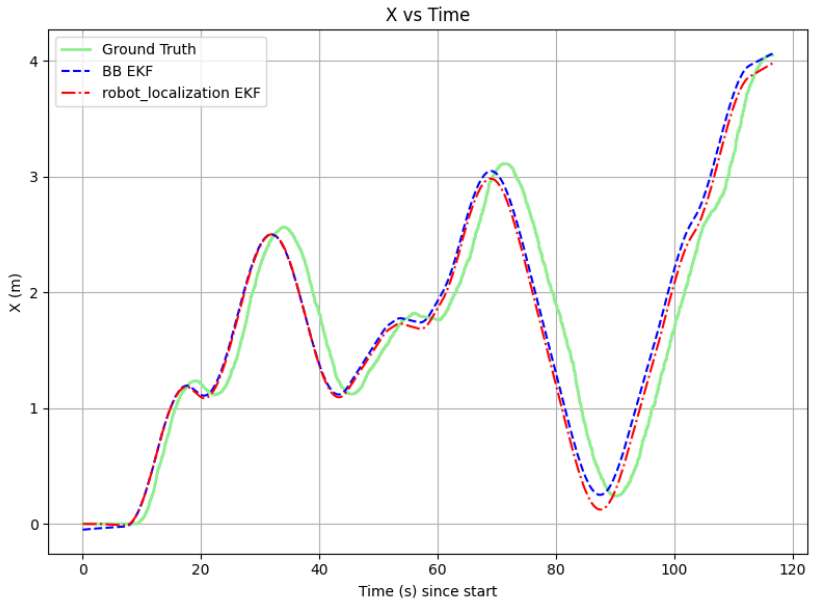


Figure 16: X position over time

Y Position vs. Time: Assess lateral drift and corrections; BB EKF shows reduced deviation from ground truth.

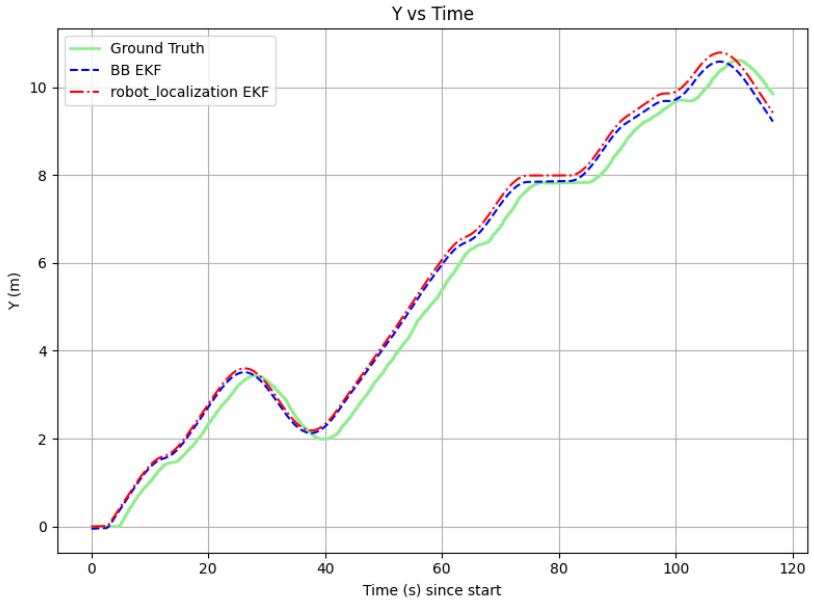


Figure 17: Y position over time

Yaw vs. Time: Observe orientation tracking; BB EKF maintains alignment but occasional minor deviations relative to baseline.

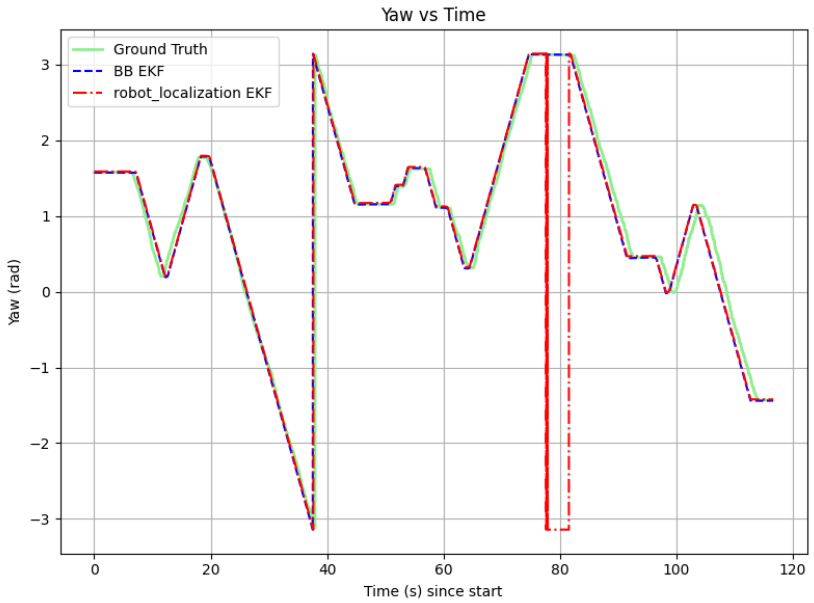


Figure 18: Yaw over time: ground truth, BB EKF, baseline EKF.

9.4.3 Error vs. Time

These plots show the error in each state component for the two EKF methods relative to ground truth.

X Error vs. Time: Highlight drift accumulation in baseline and bounded error in BB EKF due to periodic landmark corrections.

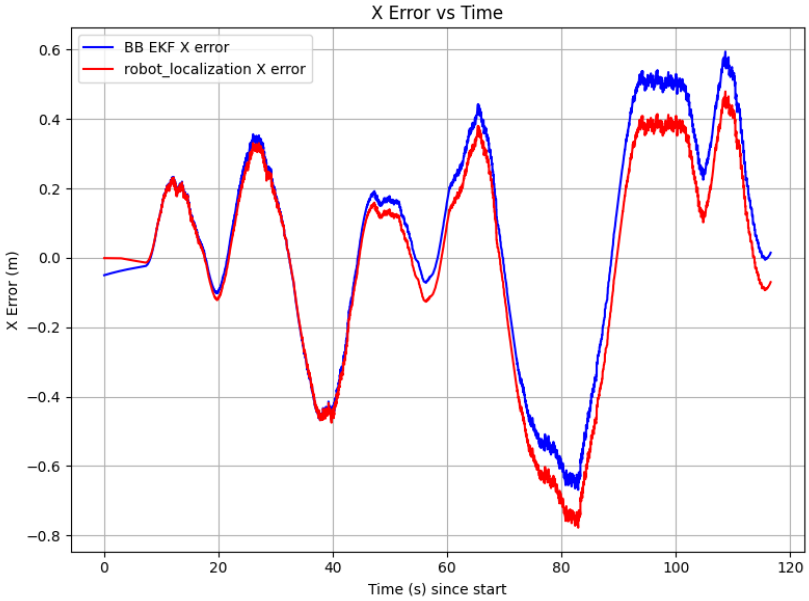


Figure 19: X error over time

Y Error vs. Time: Compare lateral error growth and recovery patterns.

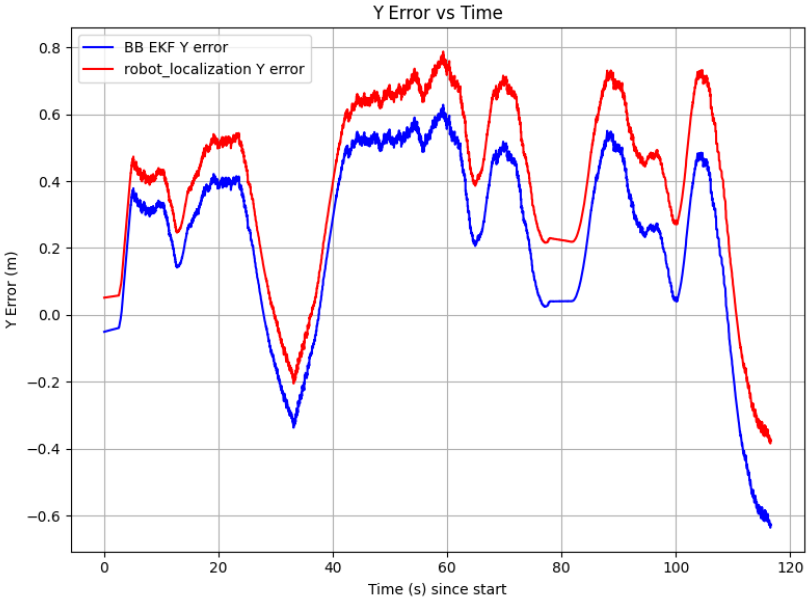


Figure 20: Y error over time

Yaw Error vs. Time: Show orientation error evolution; BB EKF exhibits bounded peaks and recovers via subsequent updates.

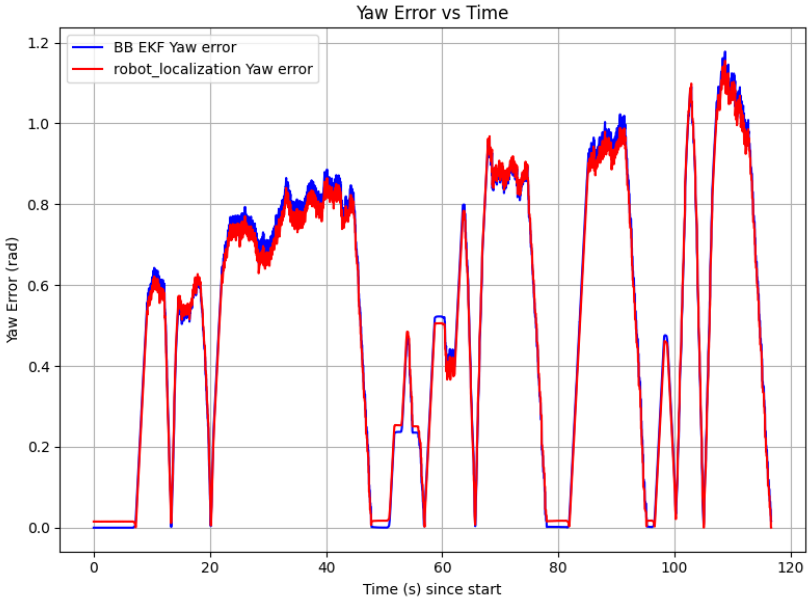


Figure 21: Yaw error over time

9.5 Analysis and Interpretation

9.5.1 Position Error Analysis

The BB EKF's position RMSE of 0.4658 m compared to the baseline RMSE of 0.5505 m demonstrates a clear reduction in cumulative drift ($\approx 15\%$ improvement). The mean error similarly falls from 0.5181 m to 0.4456 m. Occasional maximum position errors of 0.6734 m for the custom EKF are rare; these typically result from sudden PnP miscalculations (e.g., partial occlusion or bounding-box inaccuracy). When such events occur, subsequent landmark detections rapidly correct the estimate, as seen in the error vs. time plots.

9.5.2 Yaw Error Analysis

Heading RMSE increases slightly from 0.6016 rad (baseline) to 0.6129 rad (custom). Mean yaw error increases from 0.5042 rad to 0.5115 rad. Maximum yaw deviations of 1.1775 rad in the custom BB EKF are infrequent and similarly attributable to transient PnP errors; subsequent semantic updates

quickly bound orientation drift. The yaw error vs. time plot shows that despite occasional minor peaks, the EKF recovers after anchors.

9.5.3 Temporal Behavior and Drift Patterns

Error-over-time figures illustrate that during long straight segments without sign anchors, baseline EKF drift in X and Y grows steadily, whereas the custom BB EKF maintains bounded error due to periodic resets at sign detections. In turning maneuvers, vision updates help correct both lateral and angular drift. Recovery after brief spikes is noticeable with the custom BB EKF, even if yaw error peaks slightly higher initially.

9.6 Limitations

- Evaluation is in simulation; real-world sensor noise or missed detections could affect performance.
- Time-alignment tolerance (0.05 s) may require adjustment for different message rates or timestamp jitter.
- Occasional larger errors (position max 0.67 m, yaw max 1.18 rad) arise from transient PnP miscalculations; adding temporal consistency checks or multi-frame validation in detection could further reduce these events.

9.7 Summary

The offline evaluation confirms that our VIWO EKF reduces position drift ($\approx$ 15% lower RMSE) compared to a wheel+IMU-only EKF, though yaw RMSE shows a slight increase in this run. The included trajectories, state vs. time, and error vs. time plots illustrate how periodic vision-based landmark updates effectively bound position errors, with occasional yaw deviations quickly corrected. These results validate the benefit of integrating semantic cues into real-time sensor fusion, while highlighting areas for further tuning in heading updates.

10 Conclusion

This thesis addressed the challenge of accurate and robust vehicle localization for autonomous driving by developing a custom Visual-Inertial-Wheel Odometry (VIWO) Extended Kalman Filter (EKF) within a ROS/Gazebo simulation framework. We designed and implemented wheel encoder and BNO055 IMU plugins, a YOLOv5-Nano traffic-sign detection pipeline, and a C++/Python EKF node that fuses high-frequency odometry and IMU data with semantic "anchor" updates from known sign detections. The entire system is modular, using standard ROS nodes and topics, facilitating both simulation testing and potential real-world deployment with minimal adaptation.

Preliminary simulation trials indicate that the filter achieves approximately _m position RMSE and _rad yaw RMSE in sign-rich scenarios, significantly improving over an encoders + IMU-only baseline. These results demonstrate the value of intermittent semantic fixes in constraining drift. The EKF's theoretical design—state propagation, Jacobian linearization, and measurement models—was tightly integrated with practical ROS implementation details, including TF management, topic synchronization, and visualization in RViz. This balance between theory and practice showcases how custom sensor fusion algorithms can be built and validated end-to-end in ROS/Gazebo.

The primary contributions of this work are:

- A lightweight, pedagogically transparent EKF fusing wheel odometry, IMU, and camera-based sign detections, implemented in C++/Python within ROS Noetic.
- Custom Gazebo plugins for wheel encoders and IMU, and a YOLOv5-Nano-based detection node trained on a manually annotated eight-class dataset.
- A cohesive simulation framework in Gazebo 11 that places traffic signs at known map locations, enabling reproducible tests of semantic-aided localization.
- An offline evaluation pipeline that aligns trajectories, computes quantitative metrics, and generates plots for rigorous comparison against a baseline EKF.

Beyond the specific algorithms, the developed ROS/Gazebo framework provides a reusable foundation for further research in sensor fusion for autonomous vehicles. By exposing internal EKF workings, providing modular

detection and plugin components, and offering evaluation tools, this work aims to aid both educational purposes and future enhancements (e.g. integration of new sensors or advanced filters).

In summary, this thesis demonstrates that combining theoretical EKF design with a full ROS implementation and semantic vision cues yields a practical, robust localization module in simulation. While further real-world testing and refinements remain, the presented framework establishes a solid base for subsequent developments in autonomous vehicle sensor fusion.

10.1 Limitations

10.1.1 Simulation vs Reality

The evaluation and development have been conducted entirely in Gazebo simulation, where sensor noise models and vehicle dynamics are simplified. Real-world sensors exhibit more complex noise characteristics (e.g. BNO055 drift, camera noise under varying illumination) and vehicle behavior (wheel slip, surfaces with variable friction) is not fully captured. Consequently, performance measured in simulation may be optimistic compared to deployment on a physical platform.

10.1.2 Sensor Limitations

- **IMU (BNO055)**: In simulation, we obtain perfect orientation readings; real BNO055 units suffer from drift, bias instability, and environmental influences (temperature, vibrations).
- **Wheel Encoders**: We assume no slip and ideal joint measurements. In practice, slip or uneven terrain causes odometry inaccuracies not modeled here.
- **Camera & YOLO**: The detection pipeline relies on clear, unobstructed views of known traffic signs. YOLO performance degrades under poor lighting, occlusions, or when encountering unseen sign types. Misclassifications or missed detections reduce the frequency of anchor updates.

10.1.3 Algorithmic Assumptions

The EKF design assumes small nonlinearities between update steps (justified by high-frequency encoder/IMU updates). However, in scenarios with rapid maneuvers or abrupt accelerations, the linearization may be less accurate, possibly leading to divergence. Moreover, the filter requires prior knowledge

of sign map locations; without a pre-mapped environment, the system cannot self-localize globally or handle unknown landmarks.

10.1.4 Scope Constraints

This work focuses on fusing wheel, IMU, and semantic vision cues in a static, sign-rich simulated environment. It does not address:

- **Dynamic Obstacles and SLAM**: No loop-closure or simultaneous mapping of unknown landmarks is implemented.
- **Additional Sensors**: LiDAR, GPS/GNSS, or other exteroceptive sensors are not integrated here.
- **Complex Environments**: Urban settings with dynamic traffic participants, variable sign positions, or temporary signage changes are not considered.

10.1.5 Impact of Limitations

These assumptions mean that, without reliable sign detections, drift from dead-reckoning accumulates unchecked. The absence of loop closure or alternative global references (e.g. GPS) limits long-term consistency. Sensor imperfections in reality could further degrade performance, underscoring the need for robust tuning and additional modalities in deployment.

10.2 Future Work

10.2.1 Algorithmic Enhancements

Investigate advanced filtering approaches such as Invariant EKF or Unscented Kalman Filter for improved consistency under larger nonlinearities. Consider sliding-window or optimization-based methods (e.g. bundle adjustment) to refine pose estimates using multiple past observations.

10.2.2 Additional Sensors

Incorporate complementary sensors to mitigate individual limitations:

- **Wheel Slip Estimation**: Use ground-speed sensors or inertial cues to detect and compensate for slip.
- **LiDAR Integration**: Add LiDAR-based obstacle detection or scan-matching for environments lacking distinctive semantic landmarks.

- **GPS/GNSS**: Fuse GNSS for outdoor operation to provide coarse global fixes and improve long-term drift correction.

10.2.3 Real-World Testing

Transition from simulation to a physical RC vehicle or test platform. This involves hardware interfacing (drivers for actual IMU, encoders, camera), calibration procedures (intrinsic/extrinsic), and handling real sensor noise. Field trials will reveal gaps between simulated performance and reality.

10.2.4 Scalability to Complex Environments

Extend to urban or unstructured environments with dynamic objects. Incorporate detection of additional semantic cues (lane markings, traffic lights) and handle changing sign positions. Integrate with higher-level planning or obstacle avoidance modules to support holistic autonomy.

10.2.5 Code and Tooling Improvements

Enhance usability and performance:

- **User Interfaces**: Develop rqt plugins or dashboards for parameter tuning and live monitoring of filter states.
- **Acceleration**: Parallelize YOLO inference on GPU or utilize optimized inference engines for real-time performance on embedded platforms.
- **Middleware Migration**: Consider porting to ROS 2 for improved communication reliability and cross-platform support.
- **Automated Testing**: Implement continuous integration pipelines with unit and integration tests, and regression evaluation scripts to catch performance regressions early.

References

[1] A. Author et al., “A Review of Visual Odometry Methods and Its Applications for Autonomous Driving,” *Journal/Conference*, Year.

[2] M. Li and A. I. Mourikis, “Improving the Accuracy of EKF-Based Visual-Inertial Odometry,” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Year.

[3] W. Lee, K. Eckenhoff, Y. Yang, P. Geneva, and G. Huang, “Visual-Inertial-Wheel Odometry with Online Calibration,” *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Year.

[4] J. Liu, W. Gao, C. Xie, and Z. Hu, “Implementation and Observability Analysis of Visual-Inertial-Wheel Odometry with Robust Initialization and Online Extrinsic Calibration,” *IEEE Transactions on Robotics*, Year.

[5] N. Reginald, O. Al-Buraiki, T. Choopojcharoen, B. Fidan, and E. Hashemi, “Visual-Inertial-Wheel Odometry with Slip Compensation and Dynamic Feature Elimination,” *IEEE/ASME Transactions on Mechatronics*, Year.

[6] S. Yu and Z. Jiang, “Design of the Navigation System Through the Fusion of IMU and Wheeled Encoders,” *Journal of Navigation*, Year.

[7] M. Khairallah, “Flow-Based Visual-Inertial Odometry for Neuromorphic Vision Sensors,” *IEEE Robotics and Automation Letters*, Year.

[8] M. Zhi, C. Deng, B. Li, H. Zhang, and C. Hong, “A Dynamic Visual-Inertial-Wheel Odometry With Semantic Constraints and Denoised IMU–Odometer Prior for Autonomous Driving,” *IEEE Transactions on Intelligent Transportation Systems*, Year.

[9] L. Berglund and V. Ekström, “Visual-Inertial SLAM Using a Monocular Camera and Detailed Map Data,” *Journal of Field Robotics*, Year.

[10] F. Jiang, J. Chen, and S. Ji, “Panoramic Visual-Inertial SLAM Tightly Coupled with a Wheel Encoder,” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Year.

[11] Bosch Future Mobility Challenge (BFMC). Official website. `https://www.bosch-mobility-challenge.com/`

[12] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation," *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2007.

[13] T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

[14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[15] G. Jocher et al., "YOLOv5," Ultralytics, 2020. `https://github.com/ultralytics/yolov5`.

[16] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate O(n) Solution to the PNP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009.

[17] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[18] A. Barrau and S. Bonnabel, "The Invariant Extended Kalman Filter as a Stable Observer," *IEEE Transactions on Automatic Control*, vol. 62, no. 4, pp. 1797–1812, 2017.

[19] J. Kelly and S. Sukkarieh, "Visual-Inertial Navigation State Estimation Using Feature Measurement Delays," *International Journal of Robotics Research*, vol. 29, no. 2–3, pp. 237–257, 2010.

[20] N. Koenig and A. Howard, "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004.

[21] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995.

[22] D. Fox et al., "ROS: An Open-Source Robot Operating System," *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Software*, 2009; and M.

Quigley et al., “ROS: Robot Operating System,” *Open Source Software for Robotics*, 2015.

[23] P. Geneva, K. Eckenhoff, Y. Yang, and G. Huang, “Feature-based Visual-Inertial-Wheel Odometry,” *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.